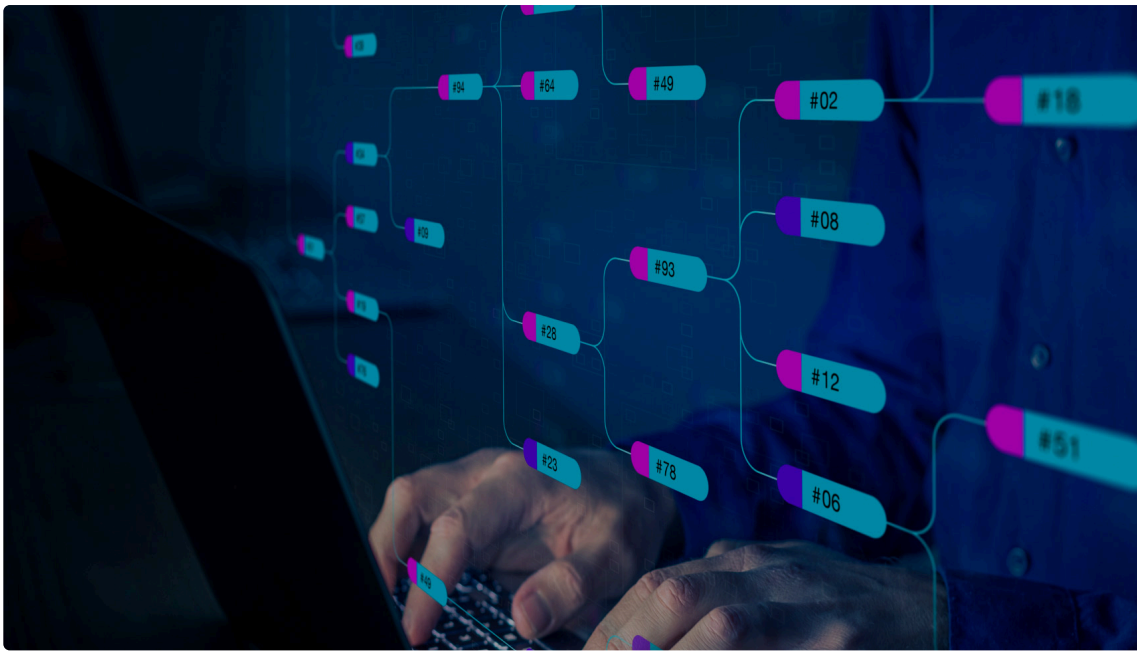


UML para modelagem do domínio

Prof. Sylvio Jorge

Apresentação

Neste tema, é apresentada uma visão geral da modelagem do domínio da aplicação com a UML por meio de modelo de casos de uso, modelo de classes, diagrama de objetos e diagrama de pacotes. Aprender as ferramentas básicas da UML para modelagem conceitual é necessário para a especificação sólida dos requisitos de um sistema, bem como para o entendimento profundo sobre o domínio do negócio.

Propósito

Preparação

Antes de iniciar este conteúdo, instale em seu computador um programa que o permita elaborar modelos sob a forma de diagramas da UML (Linguagem Unificada de Modelagem). Nossa sugestão inicial é o Astah Free Student License, e será necessário usar seu e-mail institucional para ativar a licença. Preencha os dados do formulário, envie e aguarde a liberação de sua licença em seu e-mail institucional. Ao receber a licença, siga as instruções do e-mail e instale o produto em seu computador.

Sugestões de links adicionais de programas livres para modelagem de sistemas em UML podem ser encontradas em buscas na internet.

Objetivos

Módulo 1

Requisitos funcionais de um sistema com uso de diagrama de casos de uso

Identificar requisitos funcionais de um sistema com uso de diagrama de casos de uso.

Módulo 2

Casos de uso por meio de especificações textuais

Reconhecer casos de uso por meio de especificações textuais.

Módulo 3

Diagrama de classes

Identificar classes e seus relacionamentos em um domínio de aplicação com uso de diagrama de classes.

Módulo 4

Diagramas de objetos e de pacotes

Empregar diagramas de objetos e de pacotes para apoiar a especificação de um sistema.

Introdução

Aprenderemos técnicas e notação para modelagem de requisitos e conceitos de domínio de sistemas. A Unified Modeling Language (UML) ou Linguagem de Modelagem Unificada é uma linguagem ou notação visual para especificação de sistemas orientados a objetos. Essa notação vem sendo adotada como padrão para modelagem em organizações, pois tem como objetivo principal prover um conjunto de ferramentas poderosas para análise e projeto de sistemas.

O foco deste conteúdo são os diagramas de casos de uso, diagrama de classes e objetos e diagrama de pacotes, com seus respectivos conceitos e elementos, muito úteis para arquitetos, analistas e projetistas de software na fase inicial do processo de desenvolvimento: a especificação de requisitos e modelagem conceitual do domínio.

Em Engenharia de Software, a palavra domínio denota ou agrupa um conjunto de sistemas ou de áreas com funcionalidades similares dentro dos sistemas. Assim, é possível definir domínio aplicacional como uma coleção de aplicações de software com determinadas características em comum. Logo, domínio é um conjunto de características de uma família de problemas que determinada aplicação pretende solucionar.

Como você vai observar ao longo do conteúdo, essa definição de domínio não deve ser confundida com o domínio de um atributo de classe, que é o conjunto de valores possíveis que esse atributo pode assumir.

No vídeo a seguir, você vai conferir a apresentação do tema UML para modelagem do domínio, incluindo o levantamento de requisitos e as modelagens de casos de uso, de classes, de objetos e de pacotes.

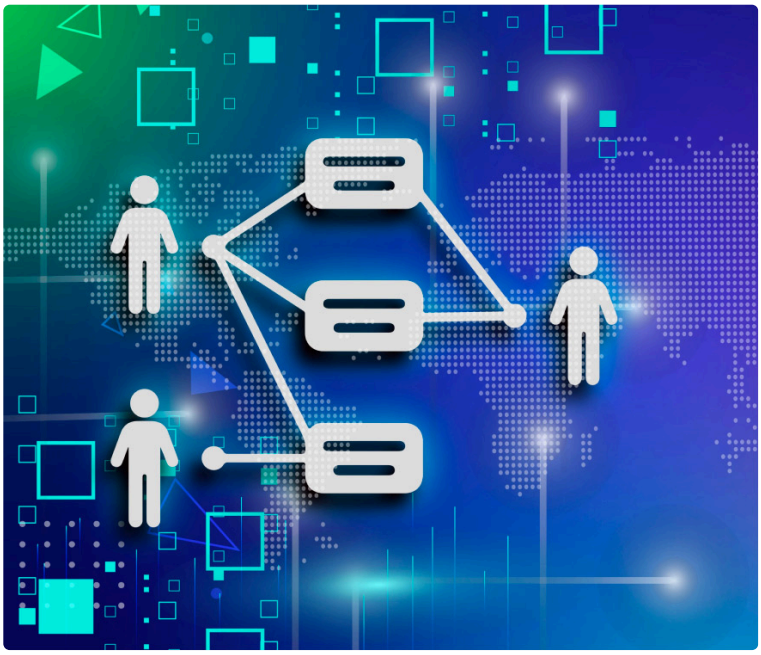
Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Material para download

Clique no botão abaixo para fazer o download do conteúdo completo em formato PDF.

Download material



1 - Requisitos funcionais de um sistema com uso de diagrama de casos de uso

Ao final deste módulo, você será capaz de identificar requisitos funcionais de um sistema com uso de diagrama de casos de uso.

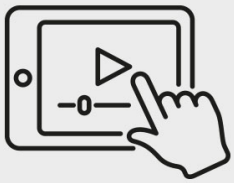
Requisitos funcionais e não funcionais

Vamos iniciar nosso estudo entendendo quais requisitos formam a base para a concepção e a implementação de soluções eficazes. Vamos, então, compreender a distinção e a importância dos requisitos funcionais e não funcionais para o sucesso na engenharia de software e no desenvolvimento de sistemas. Assim, enquanto os requisitos funcionais definem o que o sistema deve fazer, os não funcionais especificam como ele deve se comportar em termos de desempenho, segurança e usabilidade.

Lembre-se de que um conhecimento sólido sobre os tipos de requisitos prepara você a contribuir de forma significativa para projetos e equipes de desenvolvimento e para desafios futuros na área de tecnologia da informação.

Você vai acompanhar, no vídeo a seguir, a importância dos requisitos funcionais e não funcionais no desenvolvimento de um sistema, destacando que os funcionais definem o que o sistema deve fazer, enquanto os não funcionais determinam como o sistema deve operar.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



O desenvolvimento de um sistema começa pelo entendimento do problema de negócio e o levantamento de quais requisitos esse sistema de software deve satisfazer para resolvê-lo. Não existe uma definição padronizada para requisito na literatura de engenharia de software. A norma ISO/IEC/IEEE (IEEE, 1990) apresenta os seguintes enunciados para explicar o que é requisito:

> Uma condição ou capacidade do sistema solicitada por um usuário, para resolver um problema ou atingir um objetivo.

> Uma condição ou capacidade que deve ser atendida por uma solução para satisfazer um contrato, uma especificação, um

padrão ou quaisquer outros documentos formais impostos.

Documentação da representação das condições ou capacidades apresentadas nos dois anteriores.

Necessidades quantificadas e documentadas, desejos e expectativas do patrocinador, clientes e outras partes interessadas.

Os requisitos são o ponto de partida para todo o processo de desenvolvimento, sendo a base para as atividades de análise e projeto, bem como implementação e testes.

De acordo com Sommerville (2011), a área de engenharia de software faz uma distinção, em termos de nível de detalhamento de requisitos, entre requisitos de usuário e requisitos de sistema. O primeiro expressa os requisitos abstratos de alto nível, e o segundo revela uma descrição detalhada do que o sistema deve fazer. Veja outra descrição de cada um deles:

Requisitos de usuário

São declarações, em uma linguagem natural com diagramas, de quais serviços o sistema deverá fornecer a seus usuários e as restrições com as quais este deve operar (Sommerville, 2011).

Requisitos de sistema

São descrições mais detalhadas das funções, serviços e restrições operacionais do sistema de software (Sommerville, 2011).

Em uma perspectiva mais técnica, os requisitos de software são normalmente classificados como requisitos funcionais e requisitos não funcionais.

Requisitos funcionais

Veja, a seguir, como Sommerville (2011) define esses requisitos:

> “
>
> Requisitos funcionais são declarações de serviços que o sistema deve fornecer, de como o sistema deve reagir a entradas específicas e de como o sistema deve se comportar em determinadas situações.
>
> (Sommerville, 2011)

É importante ressaltar que o requisito funcional é uma funcionalidade do sistema, é algo que o sistema deve realizar para prover um resultado para o usuário. Requisitos funcionais pressupõem interação do sistema com usuários.

Veja a seguir alguns exemplos de enunciados dos chamados requisitos funcionais:

1

O usuário deve ser capaz de pesquisar por nome na base de dados disponível.

2

O sistema deve designar um identificador único (NUMERO_PEDIDO) para cada pedido do usuário.

3

O sistema deve apresentar o documento selecionado para leitura do usuário.

Perceba que, em todos os exemplos, estamos definindo atividades que o sistema deve realizar, e, em todas essas atividades, o usuário está envolvido, seja fornecendo entradas ou recebendo saídas (resultados).

Requisitos não funcionais

A especificação de um sistema precisa considerar também como esses requisitos serão entregues. Ou seja, quais características de qualidade e quais restrições desejamos impor ao sistema. Assim, definimos os chamados requisitos não funcionais. Veja o que diz Sommerville (2011):

> “
>
> Requisitos não funcionais são restrições aos serviços ou funções oferecidos pelo sistema. Incluem restrições de timing, restrições no processo de desenvolvimento e restrições impostas pelas normas. Ao contrário das características individuais ou serviços do sistema, os requisitos não funcionais, muitas vezes, aplicam-se ao sistema como um todo.
>
> **(Sommerville, 2011)**

Requisitos não funcionais (RNF) são frequentemente mais críticos do que os requisitos funcionais (RF). Deixar de atender a um RNF pode significar a inutilização de todo o sistema. Os RNF podem afetar a arquitetura geral de um sistema em vez de apenas componentes individuais. No caso de um website, um único RNF pode originar uma

série de RF relacionados, que, por sua vez, definem os serviços necessários.

Existem diversas classificações que visam a organizar e prover melhor entendimento sobre os RNF. Um exemplo desse tipo de classificação é apresentado a seguir.

Requisitos de produto

Usabilidade, eficiência, desempenho, confiança, proteção.

Requisitos organizacionais

Ambientais, operacionais, processo de desenvolvimento.

Requisitos externos

Reguladores, éticos, legais, contábeis.

Observe, a seguir, alguns exemplos de enunciados de requisitos não funcionais:

Disponibilidade do sistema

O sistema deve estar disponível durante todas as horas normais de trabalho (2ª a 6ª, 8h30 às 17h). Períodos de não operação dentro desses intervalos não podem ultrapassar 2 segundos em um dia.

Autenticação do usuário

O sistema deve requerer autenticação dos usuários para permitir acesso.

Privacidade dos usuários

O sistema deve implementar disposições de privacidade dos usuários de acordo com a Lei Geral

de Proteção de Dados.

Você pode perceber, nesses exemplos, que esses requisitos não indicam o que o sistema realiza, mas como ele entrega uma ou mais de suas funcionalidades.

Atividade 1

Qual das seguintes afirmações melhor descreve a diferença entre requisitos funcionais e não funcionais no desenvolvimento de sistemas?

A Requisitos funcionais determinam o desempenho do sistema, enquanto os não funcionais definem a interface do usuário.

B Requisitos funcionais descrevem o comportamento desejado do sistema. Já os não funcionais especificam as restrições e qualidades que o sistema deve possuir.

C Requisitos funcionais são mais importantes que os não funcionais.

D Requisitos funcionais tratam da segurança do sistema, enquanto os não funcionais lidam com a precisão dos dados.

E Requisitos funcionais são definidos após a implementação do sistema, e os não funcionais são definidos no início do projeto.

Parabéns! A alternativa B está correta.

Requisitos funcionais descrevem o comportamento específico que o sistema deve apresentar, como funcionalidades e tarefas. Já os requisitos não funcionais especificam as características e restrições do sistema, como desempenho, segurança, usabilidade e confiabilidade, que são essenciais para garantir a qualidade e a eficácia do sistema desenvolvido.

Visão de casos de uso

Ao detalhar as interações entre usuários e sistema em questão e facilitar a comunicação eficaz entre desenvolvedores, designers e stakeholders, a visão de casos de uso oferece uma compreensão abrangente dos requisitos do sistema e promove o sucesso dos projetos de software. Podemos, então, entender sua importância na área de ciência da computação e engenharia de software.

Ao dominar essa abordagem teórica, você poderá traduzir as necessidades dos usuários em funcionalidades específicas do software, garantindo que os sistemas atendam adequadamente às demandas e expectativas dos usuários finais.

Você verá, no vídeo a seguir, a visão de casos de uso da UML como um conjunto de funções que o sistema deve executar para atender às necessidades dos usuários, abordando diagrama de casos de uso, atores e caso de uso.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



A visão de casos de uso da UML apresenta o conjunto de funções (ou usos) que o sistema deve executar para atender às necessidades dos usuários. No modelo de casos de uso, o sistema é visto sob a perspectiva do usuário. A documentação dos requisitos funcionais em

um diagrama de casos de uso dá início ao processo de análise do sistema e permite entender sua abrangência e complexidade.

Diagrama de casos de uso

O objetivo do diagrama de casos de uso é apresentar uma visão geral dos requisitos funcionais do sistema e, portanto, ele é o digrama mais abstrato, flexível e simples da UML.

Esse diagrama é composto pelos seguintes elementos:

- Casos de uso
- Atores
- Relacionamentos

A Imagem 1 a seguir mostra um diagrama de casos de uso para um sistema bancário.

Imagem 1: Exemplo de diagrama de casos de uso para banco.

Os possíveis usos de um sistema são representados pelos casos de uso no diagrama. Um **caso de uso** pode estar diretamente associado a um ou mais requisitos funcionais, identificados na etapa de levantamento de requisitos do processo de desenvolvimento.

Conforme observado na Imagem 1, o caso de uso assume a forma gráfica de uma elipse contendo o seu nome no interior; nela, podemos observar quatro casos de usos representados:

- Abrir conta corrente
- Depositar dinheiro
- Sacar dinheiro
- Consultar saldo

O caso de uso é o elemento fundamental do modelo de casos de uso.

Recomendação

Embora não exista um padrão para nomear casos de uso, a boa prática recomenda utilizar uma frase verbal iniciando com verbo no infinitivo, denotando a ação prevista na funcionalidade representada pelo caso de uso.

O segundo elemento do diagrama de casos de uso é o **ator**, que é representado nesse diagrama por um boneco palito, com o nome na parte inferior.

> **O ator é quem utiliza ou interage em um caso de uso, ou seja, é uma entidade externa que interage com o sistema, mas que não faz parte dele.**

Interagir significa trocar informações com o sistema, seja fornecendo ou recebendo essas informações. Na Imagem 1, temos a representação do ator **Cliente**, que interage com os quatro casos de uso.

Exemplo

No caso de uso **Consultar Saldo**, o ator Cliente informa os dados de sua conta e recebe como resposta do sistema o valor de seu saldo.

O terceiro elemento do diagrama de casos de uso são os **Relacionamentos**, que permitem associar:

- Casos de uso com seus atores
- Casos de uso com outros casos de uso
- Atores com outros atores

A forma mais comum e obrigatória dos relacionamentos é entre atores e casos de uso, que assume a representação gráfica de segmento de reta (uma linha cheia), ligando o ator ao caso de uso no qual ele interage.

Na Imagem 1, observamos que o ator Cliente interage com os quatro casos de uso presentes nesse modelo, portanto, esses elementos gráficos estão ligados por uma reta.

A seguir, vamos detalhar cada um desses elementos.

Atores

Medeiros (2004) explica que atores atuam em casos de uso realizando atividades.

> **O ator não é uma entidade específica, mas representa um papel ou aspecto particular de uma entidade. O ator pode representar papéis executados por usuários humanos, um componente de hardware, outros**

> **sistemas com os quais o sistema em desenvolvimento se comunica. Desse modo, o nome do ator deve expressar esse papel de forma clara.**

No exemplo da Imagem 1, o ator é o papel de cliente do banco. Esse papel pode ser assumido pelos diversos clientes do banco.

Resumindo

Durante a execução de uma funcionalidade do sistema (um ou mais requisitos funcionais), o ator precisa prover entradas (informações) para o sistema e deve receber respostas dele.

Existem tipos de atores que geralmente aparecem nos sistemas. São eles:

Cargos

Empregado, cliente, gerente, vendedor, comprador etc.

Organizações e suas divisões

Fornecedor, agência reguladora, administradora de cartões, setor de vendas etc.

Outros sistemas

Sistema de vendas, sistema de cobrança, sistema de pagamento etc.

Hardware

Sensor de temperatura, leitora de código de barras etc.

O relacionamento de generalização entre atores indica que uma instância do primeiro ator interage com os mesmos tipos de instância de casos de uso do segundo. Porém, o contrário não é verdade.

Os casos de uso associados ao ator mais específico nesse relacionamento são exclusivos desse ator, que herda os casos de uso associados ao ator mais genérico. A Imagem 2, a seguir, mostra um exemplo desse tipo de relacionamento, no qual percebemos que tanto o ator Vendedor quanto o ator Supervisor interagem com o caso de uso Fazer pedido. No entanto, apenas o Supervisor participa do caso de uso Aprovar crédito. Veja:

Imagem 2: Exemplo do relacionamento de generalização entre atores,.

Normalmente, o ator inicia a sequência de interações em um caso de uso. Portanto, a forma mais fácil de começar a construir um modelo de casos de uso é identificar todos os atores.

O analista de sistemas pode observar as fontes que devem fornecer informações a serem processadas pelo sistema e para quem os resultados do processamento devam ser fornecidos. Bezerra (2015) sugere algumas perguntas úteis para guiar os analistas de sistemas:

1. Que órgãos, empresas ou pessoas utilizarão o sistema?
2. Que sistemas ou equipamentos vão se comunicar com o sistema a ser construído?
3. Alguém deve ser informado de alguma ocorrência no sistema?
4. Quem está interessado em certo requisito funcional do sistema?

A partir daí, durante a identificação dos casos de uso, o analista poderá perceber a existência de outros atores não identificados nessa etapa.

Caso de uso

Caso de uso é a descrição de uma sequência de interações entre um sistema e um ou mais atores externos a esse sistema. De acordo com Bezerra (2015):

> **Os casos de usos permitem a um observador externo ao sistema entender quais são as funcionalidades fornecidas pelo sistema e quais são os resultados externos produzidos por elas. Porém, não é possível saber como o sistema trabalha internamente para produzir esses resultados.**

Um modelo de casos de uso pode conter vários casos de uso, dependendo do tamanho e da complexidade do sistema. Veja a seguir mais algumas considerações importantes sobre esses casos:

Não se limitam à sua representação gráfica; tão importante quanto isso é a sua descrição textual. As descrições devem explicar de forma clara, simples e de fácil entendimento, o que o sistema faz, sem entrar nos detalhes de como ele deve fazê-lo. A descrição textual de um caso de uso é uma espécie de história por meio da qual contamos como um usuário realiza uma ação utilizando o sistema.

Facilitam o processo de comunicação com usuários e são utilizados para documentar a interação do sistema com eles. Desse modo, casos de uso são úteis, tanto para os profissionais envolvidos no desenvolvimento do sistema (analistas, projetistas, programadores, testadores) quanto para seus usuários e clientes.

Casos de uso podem relatar as diversas formas de utilização de uma funcionalidade do sistema. Assim, podemos definir cada uma dessas formas por meio de cenários.



O que seria um cenário?

Resposta

Pode ser entendido como a instância de execução de um caso de uso. Portanto, diversos cenários para o mesmo caso de uso podem ser escritos. Os cenários podem ser usados para a especificação dos casos de testes do sistema.

Para identificar os casos de uso, podemos pensar em dois tipos que sempre estarão presentes nos sistemas:

Casos de uso primários

Estão relacionados diretamente aos objetivos dos atores, ou seja, são as funcionalidades que agregam valor aos usuários. Bezerra (2015) sugere uma lista de perguntas para auxiliar os analistas de sistemas na descoberta dos casos de uso primários de um sistema:

- Quais são as necessidades e os objetivos de cada ator em relação ao sistema?
- Que informações o sistema deve produzir?
- O sistema deve realizar alguma ação que ocorre regularmente no tempo?
- Para cada requisito funcional, existe um caso de uso (ou mais) para atendê-lo?

Alguns casos de uso são iniciados não por um ator, mas por um evento temporal. Desse modo, normalmente o ator é o tipo de entidade que recebe a informação resultante (por exemplo, um cliente que receba uma notificação do sistema).

Casos de uso secundários

Esses casos não apresentam uma relação direta de valor para os atores, mas são imprescindíveis para o funcionamento do sistema. Por exemplo, a manutenção de cadastros (inclusão, alteração, consulta e exclusão das diversas informações que

alimentam o sistema). Um sistema bancário precisa cadastrar as informações sobre os clientes.

Atividade 2

Caso de uso é o principal elemento do modelo de casos de uso da UML. Sobre casos de uso, podemos afirmar que

A permitem a compreensão sobre como o sistema trabalha internamente para produzir os resultados.

B descrevem primordialmente requisitos não funcionais.

C assumem a forma de um retângulo com os cantos arredondados no diagrama de casos de uso.

D fornecem uma perspectiva do sistema do ponto de vista de um observador externo.

E descrevem as entidades do sistema.

Parabéns! A alternativa D está correta.

Casos de uso descrevem funcionalidades do sistema (requisitos funcionais) na perspectiva dos seus usuários.

Relacionamentos

Os relacionamentos são responsáveis por estabelecer as associações entre atores e casos de uso. Todos os atores devem estar associados explicitamente aos casos de uso com os quais interagem. Além disso, podemos estabelecer relacionamentos entre os casos de uso ou entre os atores do sistema.

O modelo de casos de uso permite os seguintes tipos de relacionamentos: comunicação, inclusão, extensão e generalização.

Neste vídeo, você vai conferir a utilização dos relacionamentos no diagrama de casos de uso, que são: de comunicação, inclusão, extensão e generalização, responsáveis por estabelecer as associações entre atores e casos de uso.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Relacionamento de comunicação

É uma linha simples que mostra a que caso de uso determinado ator está associado, ou seja, esse ator interage com o sistema por meio do caso de uso.

Um ator pode se relacionar com mais de um caso de uso, e todos os atores devem estar associados a pelo menos um caso de uso. Esse relacionamento foi mostrado na Imagem 1 apresentada anteriormente.

Atenção!

Um erro comum na elaboração de diagramas de casos de uso é estabelecer relacionamentos de comunicação entre casos de uso. Não existe troca de informações entre casos de uso, uma vez que são funcionalidades independentes entre si.

Já os relacionamentos de inclusão (include), extensão (extend) e generalização são estabelecidos entre casos de uso.

Relacionamento de inclusão

Relacionamento de inclusão tem como objetivo principal a reutilização. Veja a seguir como pode ser utilizada:

Repetição de passos de execução		Criação e inclusão de um caso de uso
Suponha que uma parte dos passos de execução de um caso de uso se repita em diversos outros casos de uso.	→	Podemos criar um caso de uso separado e “incluí-lo” nos demais.

A inclusão funciona de forma semelhante à chamada de uma função em um programa. A representação gráfica do relacionamento de inclusão é uma linha pontilhada com o estereótipo <<include>> apontando do caso de uso inclusor para o incluído.

A Imagem 3 mostra um exemplo desse tipo de relacionamento:

Imagem 3: Exemplo do relacionamento de inclusão entre casos de uso.

Na Imagem 3, o ator Médico participa de dois casos de uso: Fazer pedido de exame e Fazer pedido de consulta. Em ambos os casos de uso, é necessário realizar os passos do caso de uso Solicitar prontuário. Portanto, este caso de uso é incluído nos dois primeiros.

Relacionamento de extensão

Tem como objetivo representar situações em que diferentes sequências de passos possam ser inseridas no mesmo caso de uso. Ou seja, dependendo de uma condição, o caso de uso segue um caminho diferente. Desse modo, um caso de uso estende outro, permitindo um comportamento eventual.

Comentário

A existência do caso de uso estendido deve ser independente da existência de quaisquer casos de uso que o estendam.

A Imagem 4 mostra um relacionamento de extensão. Veja:

Imagem 4: Exemplo do relacionamento de extensão entre casos de uso.

Observe na Imagem 4 que o ator Cliente interage com o caso de uso Realizar compra. O caso de uso Validar limite de crédito pode ser necessário nesse contexto, portanto, é estendido para Realizar compra. Ambos os casos de uso representam funcionalidades independentes no sistema.

Relacionamento de generalização

Pode existir entre dois casos de uso ou entre dois atores (conforme vimos em Ator). Esse relacionamento permite que um caso de uso (mais específico) herde características de outro (mais genérico).

Veja o que acontece na generalização entre casos de uso:

Herança de um caso de uso

Quando um caso de uso herda de outro, significa que as sequências de passos de execução do caso de uso mais genérico (pai) valem também para o mais específico (filho).

Caso de uso herdeiro

O caso de uso herdeiro participa de qualquer associação do qual o caso de uso mais genérico participe. Isso significa que um ator que interage com o caso de uso pai pode também interagir em qualquer caso de uso filho.

A Imagem 5 mostra um relacionamento de generalização:

Imagem 5: Exemplo do relacionamento de generalização entre casos de uso.

Como podemos observar na Imagem 5, o caso de uso Cadastrar empregado executa uma sequência de passos de cadastro para qualquer funcionário.

Já os casos de uso mais específicos (Cadastrar vendedor e Cadastrar supervisor) herdam esses passos e ainda possuem passos específicos para os tipos de funcionários Vendedor e Supervisor, respectivamente.

Os tipos de relacionamento entre casos de uso abordados permitem manter a descrição dos casos de uso mais simples e sem repetições. Assim, veja como eles devem ser usados:

Inclusão

A inclusão deve ser usada quando um comportamento se repetir em mais de um caso de uso.

Extensão

A extensão é normalmente usada quando um comportamento que não é executado em todas as instâncias de um caso de uso deve ser descrito.

Generalização

A generalização deve ser aplicada em uma situação em que dois ou mais casos de uso possuam comportamentos comuns.

Atividade 3

Os relacionamentos estabelecem as associações entre atores e casos de uso. Qual dos seguintes relacionamentos entre casos de uso é realizado para representar a reutilização de passos de execução em diferentes casos de uso?

A Relacionamento de comunicação.

B Relacionamento de inclusão.

C Relacionamento de extensão.

D Relacionamento de generalização.

E Relacionamento de agregação.

Parabéns! A alternativa B está correta.

O principal objetivo do relacionamento de inclusão (include) é a reutilização dos passos de execução em diferentes casos de uso. Quando uma parte dos passos de execução de um caso de uso se repete em diversos outros casos de uso, pode-se criar um caso de uso separado e incluí-lo nos demais. Isso funciona de forma semelhante à chamada de uma função em um programa e é representado graficamente por uma linha pontilhada com o estereótipo <> apontando do caso de uso inclusor para o incluído.

Modelagem de casos de uso na prática

Aprender a modelagem de casos de uso na prática dá a você acesso a uma ferramenta interessante para traduzir os requisitos do cliente em funcionalidades tangíveis do sistema. Isso facilita a comunicação entre desenvolvedores, designers e stakeholders e garante que o produto final atenda às necessidades reais dos usuários.

Para entender e treinar a confecção de um diagrama de casos de uso, vamos usar como minimundo uma locadora de veículos. Após entrevistar o responsável, chegou-se às seguintes conclusões:

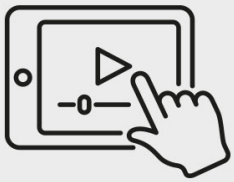
1. Cada cliente só pode alugar um veículo de cada vez, e o aluguel só pode ser feito quando um funcionário acessa o sistema.
2. Um cliente pode consultar remotamente os veículos disponíveis, que são categorizados por tipo.
3. O cliente pode prorrogar remotamente o aluguel sem que seja necessária a intervenção de algum funcionário.
4. A devolução do veículo também só pode ser feita por um funcionário acessando o sistema, e não precisa ser o mesmo que registrou o aluguel.

5. Três tipos de veículos estão disponíveis: luxo, normal e popular.
6. Os veículos de um mesmo tipo têm preços e características similares.
7. Quando a locadora adquire um veículo, um funcionário deve cadastrá-lo no sistema.

Modelagem de casos de uso na prática

No vídeo a seguir, você assistirá a uma apresentação que permitirá a prática da modelagem de casos de uso a partir de determinado minimundo e utilizando a UML. A modelagem de casos de uso é uma técnica que descreve as interações entre usuários e um sistema, identificando suas principais funcionalidades.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Agora vamos usar o diagrama de casos de uso para representar a descrição anterior do minimundo locadora de automóveis. O diagrama é criado em três passos:

1. identificação dos atores
2. identificação dos casos de uso
3. desenho do diagrama

A partir da descrição do minimundo, vamos identificar os atores e os casos de uso. Para identificar os atores, temos que perceber quem interage com o sistema. O texto resultante da entrevista mostra que apenas dois atores interagem com o sistema: cliente (acesso remoto) e funcionário. O próximo passo, após a identificação dos atores, é definir quais são as suas interações com o sistema. Cada interação corresponde a um caso de uso.

- No item 1 fica clara a existência do caso de uso alugar veículo, que é realizado pelo ator funcionário.
- Os itens 2 e 3 mostram a existência de dois casos de uso remotos: consultar veículos e prorrogar aluguel. Ambos os casos de uso são executados pelo ator cliente.
- O item 4 indica a existência do caso de uso devolver veículo, e o ator desse caso de uso é o funcionário.
- Os itens 5 e 6 não indicam nenhum novo caso de uso.
- Mas o item 7 introduz o caso de uso cadastrar novo veículo, que é realizado pelo funcionário.

Uma vez identificados os atores e os casos de uso, o próximo passo é desenhar diagrama de casos de uso da imagem com os cinco casos de uso e os dois atores.

Imagem 6: Diagrama de casos de uso.

Atividade 4

Considerando o minimundo da locadora de veículos, caso o dono da locadora queira instituir um auxílio produtividade para os funcionários, quais casos de uso permitem medir a produtividade dos funcionários da locadora?

A Alugar veículo, prorrogar aluguel e cadastrar novo veículo.

B Consultar veículo, prorrogar aluguel e cadastrar novo veículo.

C Consultar veículo, encerrar aluguel e cadastrar veículo.

D Encerrar aluguel, cadastrar novo veículo e prorrogar aluguel.

E Alugar veículo, encerrar aluguel e cadastrar novo veículo.

Parabéns! A alternativa E está correta.

Os três casos de uso são os realizados pelo ator funcionário. Se o ator sempre se identificar ao realizar cada um dos casos de uso, o sistema armazenará o número de vezes em que cada um utilizou cada caso de uso.



2 - Casos de uso por meio de especificações textuais

Ao final deste módulo, você será capaz de reconhecer casos de uso por meio de especificações textuais.

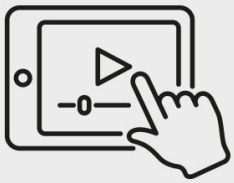
Especificação textual de casos de uso

Na engenharia de software, para descrevermos detalhadamente a interação entre atores e sistema, precisamos utilizar a especificação textual de casos de uso. Geralmente escrita em linguagem natural, ela delineia os requisitos funcionais do sistema, detalhando as ações que os atores executam e as respectivas respostas do sistema. Uma estrutura útil inclui campos como identificador único, nome, objetivo, atores, pré-condições, cenário principal, cenários alternativos, pós-condições e regras de negócio.

Uma especificação textual bem elaborada possibilita uma base clara e compreensível para o desenvolvimento do software, promovendo a comunicação eficaz entre equipes e stakeholders durante todo o ciclo de vida do projeto.

Você vai conferir, neste vídeo, os casos de uso incluídos no diagrama de casos de uso, que devem ser especificados em um formato textual para complementar a representação gráfica. Verá também o cenário principal e os alternativos de casos de uso.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Especificação e formatos de casos de uso

Os casos de uso incluídos pelos analistas no diagrama de casos de uso devem ser especificados em um formato de narrativa textual, para complementar a representação gráfica sob forma de diagramas. Desse modo, o modelo de casos de uso é formado pelo diagrama e pelo conjunto de todas as descrições dos casos de uso.

> **A descrição textual permite o entendimento mais preciso de qual funcionalidade do sistema trata aquele caso de uso. Ela serve como ponto de partida para construção de outros modelos, tais como, modelo de classes, ainda na fase de análise, e o diagrama de sequência, na fase de projeto do desenvolvimento do sistema.**

A UML não propõe uma descrição formal de casos de uso e não existe uma forma padronizada para a escrita dessa especificação. Os analistas e, muitas vezes, as empresas definem seus padrões com base em

exemplos, boas práticas e necessidades particulares de cada projeto de desenvolvimento. Assim, as especificações podem ser:

Simples

As especificações simples abstraem os detalhes da execução do caso de uso.

Complexas

As especificações mais complexas incluem todo o passo a passo a ser seguido para obter os resultados esperados.

Uma vez que não existe um padrão definido pela UML, a descrição textual de um caso de uso pode ser feita em diferentes formatos. O que há em comum em todos esses formatos é a **forma de expressão narrativa**.

O texto de um caso de uso é uma história que conta as ações ou a sequência de passos de um ou mais atores e do sistema para atingir determinado objetivo.

Recomendação

A descrição de um caso de uso deve ser clara, livre de jargões e termos técnicos, portanto, de fácil entendimento não só para a equipe de desenvolvimento, mas para os usuários finais, pois o modelo de casos de uso é lido e validado por todos os envolvidos.

Essa narrativa pode ser escrita em um parágrafo contínuo, em passos numerados ou em uma tabela. Veja cada uma dessas formas adiante.

Formato contínuo

Neste formato, a narrativa é um texto livre. Como exemplo, considere o caso de uso Consultar saldo já apresentado anteriormente e que retomaremos mais uma vez a seguir.

Imagem 1: Exemplo de diagrama de casos de uso para banco.

Uma especificação em formato contínuo para esse caso de uso seria:

Este caso de uso inicia quando o cliente chega ao caixa eletrônico e insere seu cartão. O sistema requisita a senha do cliente. Após o cliente fornecer sua senha e esta ser validada, o Sistema exibe as opções de operações possíveis. O cliente opta por consultar saldo. O sistema exibe o saldo da conta e o caso de uso termina.

Formato numerado

No formato numerado, a descrição do caso de uso da Imagem 1 seria:

1. O cliente insere cartão no caixa eletrônico.
2. O sistema solicita a senha.
3. O cliente digita senha.
4. O sistema valida a senha e exibe menu de operações disponíveis.
5. O cliente seleciona a opção consultar o saldo.
6. O sistema exibe o saldo da conta.

Formato tabular

No formato tabular, a descrição do mesmo caso tratado anteriormente seria uma tabela contendo uma coluna para cada ator e uma coluna para o sistema, da seguinte forma:

Cliente	Sistema
Insere cartão no caixa eletrônico.	
	Solicita a senha.
Digita senha.	
	Valida a senha e exibe menu de operações disponíveis.
Seleciona a opção consultar o saldo.	
	Exibe o saldo da conta.

Tabela: Formato tabular para o exemplo de diagrama de casos de uso.
Flavia Maria Santoro.

Cenário principal e cenários alternativos

A especificação de um caso de uso descreve a sequência de passos de interação entre o sistema e seus atores. A descrição do exemplo do caso de uso Abrir conta corrente, na Imagem 1, mostra uma sequência,

um fluxo ou passo a passo mais usual, ou seja, aquela em que acontece exatamente o que era esperado nessa interação.

Essa sequência é chamada de cenário (ou fluxo) principal do caso de uso. Nesse cenário, o ator consegue atingir seu objetivo seguindo o caminho “feliz”; em outras palavras, nada de excepcional acontece. De acordo com Lima (2005), é o fluxo mais óbvio, no qual todas as ações são bem-sucedidas.

Comentário

Não se esqueça de que cada caso de uso possui apenas um cenário principal e deve ser descrito de forma clara e concisa, sem uso de jargão computacional. Além disso, devem ser escritos do ponto de vista do usuário e, sempre que possível, com os termos que os usuários possam compreender.

Agora, imagine a seguinte situação em nosso exemplo: O Cliente digita a senha errada. O que aconteceria no fluxo descrito?

Nada, pois esse problema não é previsto nesse cenário!

Para essa situação e algumas outras, o modelo de casos de uso prevê a especificação dos **cenários alternativos** e **cenários de exceção**. Entenda a seguir esses cenários:

Cenários alternativos

Nesta situação, o ator pode escolher formas diferentes para chegar ao seu objetivo. Portanto, deve ser possível, apesar de não obrigatório, especificar cenários alternativos. Em nosso exemplo da consulta de saldo, o cliente poderia optar por imprimir o saldo em vez de consultá-lo diretamente. **Um cenário alternativo não é uma exceção, é outra forma de fazer a mesma coisa!**

Cenários de exceção

Devem ser explicitados exceções, problemas ou possíveis erros. Um cenário de exceção representa uma anomalia não prevista no cenário principal, como o cliente digitar sua senha errada. O analista deve escrever cenários para tratar esse e outros desvios do curso normal de um caso de uso, ou mesmo cancelar a realização do caso de uso em questão.

De acordo com Bezerra (2015), esses cenários possuem as seguintes características:

1

Representam erros de operação no cenário principal.

2

Não têm sentido fora do contexto do caso de uso no qual ocorre (ou seja, não são casos de uso em si).

3

Precisam indicar em que passo o caso de uso continua ou, conforme for, indicar que o caso de uso está finalizado.

No exemplo do caso de uso Abrir conta corrente, um cenário de exceção para a situação em que o Cliente digitasse sua senha errada poderia ser especificado da seguinte forma:

Cenário principal

1. O cliente insere cartão no caixa eletrônico.
2. O sistema solicita a senha.
3. O cliente digita senha.
4. O sistema valida a senha e exibe menu de operações disponíveis.
5. O cliente seleciona a opção consultar o saldo.
6. O sistema exibe o saldo da conta.

Cenário de exceção “Senha incorreta”

No passo 4, caso a senha seja incorreta:

4a. O sistema informa que a senha está incorreta e solicita que o cliente digite novamente.

4b. O caso de uso retorna ao passo 3.

Observe que o cenário de exceção indica o ponto do cenário principal no qual o problema pode ocorrer e o ponto exato para onde o fluxo deve retornar. O analista deve especificar todas as possíveis exceções ou problemas usando esses tipos de cenários.

Atividade 1

Durante a execução de um caso de uso, podem ocorrer exceções. Assinale a opção que descreve como essas situações devem ser tratadas no modelo de casos de uso:

A O analista deve especificar cenários de exceção.

B O analista deve desconsiderar esse tipo de situação.

C O analista deve criar um diagrama de tratamento de exceções.

D O analista deve solicitar ao usuário que não se desvie do cenário principal do caso de uso.

E O analista deve incluir um tratamento para a exceção no cenário principal.

Parabéns! A alternativa A está correta.

A forma correta de tratar exceções no modelo de casos de uso é especificar cenários de exceção que, diferentemente dos cenários alternativos, representam situações de anomalias que rompem o fluxo do cenário principal.

Especificação de casos de uso

Embora a UML não defina um padrão específico para a especificação de casos de uso, diversas abordagens práticas podem ser adotadas. Assim, empresas e equipes frequentemente estabelecem seus próprios padrões, mantendo a qualidade dos documentos de requisitos. Além disso, a UML não define uma forma padrão para casos de uso de inclusão ou extensão, mas é comum fazermos referência ao caso incluído/estendido na descrição do caso que o inclui/estende.

Você verá, neste vídeo, como é estruturada a especificação de casos de uso, e qual a sua finalidade com inclusão e extensão.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Estrutura de especificação de casos de uso

Como a UML não define um padrão para especificação de casos de uso, você já observou que existem algumas formas práticas de apresentar a narrativa ou o passo a passo de execução do caso de uso.

No entanto, é comum as empresas ou as equipes de desenvolvimento definirem seus próprios padrões de estrutura e os tipos de informação a serem apresentados, de modo a manter a qualidade dos documentos de requisitos de software. Nesse sentido, você encontrará diversas sugestões de modelos em livros ou sites na internet.

Um exemplo bastante útil é a estrutura descrita por Bezerra (2015). Veja, a seguir, uma adaptação dessa estrutura, que destaca os campos que poderiam compor a descrição de um caso de uso. Você pode tomar como base essa proposta e criar um padrão para seus projetos.

1. Identificador: código único para cada caso de uso que permite fazer a rastreabilidade com outros elementos e modelos. Por exemplo, com casos de teste e regras de negócio, UC01.
2. Nome: cada caso de uso tem um nome único que o identifica; é o nome que aparece no diagrama de casos de usos. O nome do caso de uso é uma frase verbal iniciada por verbo no infinitivo. É importante ter cuidado com coerência e clareza ao nomear um caso de uso. Por exemplo, o nome do caso de uso UC01 é “Criar conta corrente”.
3. Objetivo: declaração sucinta do objetivo do ator no caso de uso. Para o caso de uso Criar conta corrente, o objetivo seria: “O objetivo deste caso de uso é permitir que o Cliente do banco crie uma nova conta corrente”.
4. Ator primário: nome do ator que inicia o caso de uso ou ator que recebe o resultado do caso de uso; um caso de uso possui apenas um ator primário. Para o caso de uso Criar conta corrente, o ator primário seria: “Cliente”.
5. Atores secundários: nomes dos demais atores que atuam no caso de uso. Para o caso de uso Criar conta corrente, um ator secundário seria: “Gerente da agência”.

6. Pré-condições: condições assumidas como verdadeiras para que o caso de uso tenha início; um caso de uso pode conter zero ou mais pré-condições. Por exemplo, em um caso de uso Criar conta corrente, uma pré-condição poderia ser “Cliente cadastrado com CPF válido”.

7. Cenário (ou fluxo) principal: sequência de passos que normalmente acontece quando o caso de uso é utilizado com sucesso; toda descrição de caso de uso deve ter um fluxo principal.

8. Cenários (ou fluxos) alternativos e de exceção: sequência de passos que apresentam formas de execução diferentes da descrita no fluxo principal ou situações de exceções e erros que devem ser tratadas pelo sistema. Não há obrigatoriedade de existirem fluxos alternativos ou de exceção em um caso de uso.

9. Pós-condições: estado do sistema após o caso de uso ter sido executado; tipicamente, uma pós-condição descreve que uma informação foi alterada, excluída ou inserida no sistema. Uma boa prática é usar verbo no particípio passado para descrever pós-condições. Por exemplo, em um caso de uso Criar conta corrente, uma pós-condição seria “conta corrente criada”.

10. Regras de negócio: a descrição de um caso de uso pode fazer referência a uma ou mais regras de negócio. Aprenda mais sobre Regras de Negócio em Explore +.

Especificação de casos de uso com inclusão e extensão

A UML também não define uma forma padrão para a escrita de casos de uso de inclusão ou extensão. No entanto, geralmente fazemos referência ao caso de uso incluído/estendido na descrição do caso de uso que o inclui/estende. Veja como pode ser a escrita desses casos:

Casos de uso com inclusão

Para a situação dos casos de uso de inclusão, a seguinte sintaxe pode ser usada: Include (nome do caso de uso incluso) no ponto da descrição da sequência de interações em que o caso de uso deve ser inserido.

Casos de uso com extensão

Para essas situações, devemos lembrar que o caso de uso estendido existe independentemente de outros casos de uso

que o estendam. Ele possui a descrição completa de sua sequência de passos. Se o ator escolhe acessar o caso de uso estendido, ele executa o fluxo de ações desse caso de uso e retorna para o ponto do cenário em que estava antes.

A especificação dos pontos de extensão em um caso de uso é feita normalmente dentro da descrição textual do caso de uso extensor. A descrição de um ponto de extensão deve indicar em que ponto (ou pontos) do caso de uso estendido pode-se inserir o comportamento do extensor, e podem existir vários pontos no estendido em que o extensor pode ser ativado.

Atenção!

Além da inclusão e extensão, um relacionamento de generalização requer uma forma de especificação, na descrição do caso de uso herdeiro, de quais passos do caso de uso pai estão sendo redefinidos. O analista pode criar marcadores no caso de uso pai. Esses marcadores podem, então, ser referenciados na descrição do caso de uso filho para especificar que passos estão sendo redefinidos ou onde está sendo definida uma extensão do comportamento do pai.

Atividade 2

O modelo de casos de uso complementa a especificação gráfica, realizada por meio do diagrama de casos de uso com uma especificação textual. Qual das seguintes afirmações sobre essa especificação textual está correta?

A — A UML define um padrão rígido para a especificação de casos de uso, limitando as opções das empresas e equipes de desenvolvimento.

B — A estrutura que inclui identificador único, nome, objetivo, atores, pré-condições, cenário principal, cenários alternativos, pós-condições e regras de

negócio é a única forma aceita para especificar casos de uso em projetos de software.

C As pré-condições em um caso de uso são condições que devem ser garantidas antes da execução do caso de uso.

D Os atores secundários em um caso de uso são aqueles que iniciam o caso de uso ou recebem o resultado dele.

E A inclusão e extensão são os únicos relacionamentos entre casos de uso reconhecidos pela UML.

Parabéns! A alternativa C está correta.

As pré-condições em um caso de uso são condições assumidas como verdadeiras para que o caso de uso tenha início. Essas condições devem ser verdadeiras antes da execução do caso de uso, e não após.

Especificação textual de casos de uso na prática

Praticar a descrição textual dos casos de uso dá a capacidade de capturar os passos que representam a interação entre o ator e o caso de uso. Na descrição, os passos indicam as ações dos dois agentes envolvidos: ator e sistema. A descrição também enfatiza as informações tratadas, o que vai proporcionar a identificação das classes candidatas e de seus atributos.

Você acompanhará, neste vídeo, a prática da especificação textual de casos de uso, que permite detalhar as interações entre os usuários e o sistema de maneira escrita, incluindo os passos necessários para alcançar um objetivo específico a partir de um determinado minimundo e utilizando a UML.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

A seguir será apresentada a descrição do caso de uso alugar veículo. Ator funcionário executa caso de uso alugar veículo, de acordo com os seguintes passos:

1. Funcionário inicia caso de uso.
2. Sistema solicita código do funcionário.
3. Funcionário fornece seu código.
4. Sistema valida e solicita dados do cliente.
5. Funcionário fornece a carteira nacional de habilitação (CNH) do cliente.
6. Sistema verifica CNH.
    6.1. Se CNH já for cadastrada, o sistema disponibiliza demais dados (a saber: nome, telefone e endereço) e vai para o passo 7.
    6.2. Se não for, o funcionário fornece demais dados (por exemplo: nome, telefone e endereço).
    6.3. Sistema armazena informações.
7. Sistema disponibiliza tipos de veículo com automóveis à disposição para aluguel.
8. Funcionário seleciona tipo de veículo.
9. Sistema disponibiliza veículos daquele tipo.
10. Funcionário seleciona veículo, informando data de início e de fim de aluguel.
11. Sistema armazena dados do aluguel.

12. Caso de uso é encerrado.

Cenário de exceção 1

No passo 4, o código fornecido pelo funcionário no passo 3 não é validado pelo sistema:

- 4a. Sistema não valida código do funcionário.
- 4b. Se for não for a terceira tentativa, o sistema retorna ao passo 3.
- 4c. Se for a terceira tentativa, é executado o passo 12.

Cenário de exceção 2

No passo 7 não há veículos disponíveis para aluguel:

- 7a. Passo 12 é executado.

Atividade 3

Considerando o minimundo locadora de veículos, qual das seguintes opções representa um passo válido de um dos casos de uso?

A O cliente reserva um veículo sem verificar a disponibilidade.

B O funcionário registra o aluguel de um veículo sem autenticar-se no sistema.

C O cliente acessa o sistema e adiciona um novo tipo de veículo.

D O funcionário cadastra um novo veículo no sistema após a sua aquisição.

E O cliente prorroga o aluguel de um veículo entrando em contato por telefone.

Parabéns! A alternativa D está correta.

De acordo com as conclusões obtidas após a entrevista com o responsável da locadora de veículos, a informação de que "quando a locadora adquire um veículo, um funcionário deve cadastrá-lo no sistema" está presente. As outras opções não refletem adequadamente os processos descritos nas conclusões.



3 - Diagrama de classes

Ao final deste módulo, você será capaz de identificar classes e seus relacionamentos em um domínio de aplicação com uso de diagrama de classes.

Conceito e elementos de uma classe

Qualquer pessoa que busque compreender a programação orientada a objetos precisa ter entendimento sólido dos conceitos e elementos de uma classe. Portanto, estudar esse assunto capacita quem deseja projetar sistemas robustos e flexíveis, pois promove a reutilização e a manutenção do código.

Classe é a estrutura fundamental dos diagramas de classes, pois representa um modelo ou tipo de objeto que encapsula dados e operações relacionadas. Dentro de uma classe, os elementos principais incluem atributos (ou variáveis de instância), que descrevem as características do objeto, e métodos (ou funções de membro), que definem seu comportamento.

Neste vídeo, você verá que uma classe inclui a descrição de atributos (características) e serviços (ações) comuns a um grupo de objetos, apresentando o conceito de classe e seus elementos essenciais.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



A abordagem de desenvolvimento de sistemas orientada a objetos assume que um sistema é uma coleção de entidades – os objetos – que se comunicam e realizam ações. A interação entre objetos é a forma de execução de uma funcionalidade do software.

Essa abordagem se baseia nos seguintes princípios listados por Bezerra (2015):

1

Qualquer coisa representa um objeto.

2

Objetos realizam tarefas por meio da requisição de serviços a outros objetos.

3

Cada objeto pertence a uma classe, ou seja, classe é uma

entidade que agrupa e abstrai objetos similares.

4

Uma classe funciona com repositório para comportamento associado aos objetos instanciados a partir dela.

5

Classes são organizadas em hierarquias.

Entenda, a seguir, algumas definições e termos utilizados anteriormente:

UML

A UML é a linguagem padrão que permite a elaboração dos diversos modelos e das perspectivas de um sistema na abordagem da orientação a objetos.

Classes

Classes (e seus objetos) são os componentes básicos dessa abordagem.

Objetos

Objetos são entidades do mundo real que podemos observar. Por exemplo, um um carro, uma fruta, uma empresa, um empregado da empresa, um concerto de música, um restaurante, entre outros.

Quando pensamos nesses objetos em termos de grupos de coisas, por exemplo, cada pessoa que assiste a aulas em uma turma é um objeto diferente, mas, se abstrairmos esses objetos, observando suas características comuns, criamos o conceito de Aluno, o qual é denominado, então, de Classe. Uma classe de objetos com

caraterísticas comuns, tais como: estão matriculados em uma turma e assistem a aulas.

> **Uma classe pode ser definida como uma descrição de atributos (características) e serviços (ações) comuns a um grupo de objetos.**

Bezerra (2015) ressalta que uma classe pode ser entendida como uma espécie de molde, a partir do qual objetos podem ser construídos. Portanto, é comum afirmarmos que um objeto é uma instância de uma classe. Cada objeto do mundo real é único, uma classe abstrai o tipo de objeto e permite que novos objetos sejam criados a partir desse modelo.

Exemplo

Se uma nova pessoa se matricula na turma, ela é um novo objeto instanciado a partir da classe Aluno.

Quando estamos tratando da modelagem de um sistema, somente um subconjunto de características do grupo de objetos pode ser relevante. Portanto, uma classe representa uma abstração das características relevantes do mundo real para aquele conjunto de objetos.

Voltando aos nossos alunos, as características relevantes desses objetos, para um sistema acadêmico, poderiam ser número de matrícula, idade e estilo de aprendizagem; mas outras, tais como o esporte favorito ou número do calçado, talvez não façam parte do interesse desse sistema.

A estrutura de uma classe é composta de atributos e de operações. Veja o que cada um representa a seguir:

Atributos

Os atributos descrevem os dados que deverão ser armazenados pelos objetos dessa classe (suas características). Cada atributo de uma classe é de um tipo de dados e possui um conjunto de valores que esse atributo pode assumir, chamado de domínio.

Operações

As operações apresentam as ações que os objetos de uma classe podem realizar. Como são ações, costumam ser nomeadas com uso de um verbo e um complemento, e terminam

com um par de parênteses (funções em linguagens de programação). Ao contrário dos atributos, que para cada objeto têm o seu próprio valor, os objetos de mesma classe compartilham as suas operações. Em outras palavras, todos os objetos de uma classe realizam as mesmas ações.

Veja um exemplo de atributos e operações na Imagem 1 a seguir:

Imagem 1: Exemplo de classe e seus atributos e operações,

Observamos, na Imagem 1, que a representação de uma classe é feita por meio de um retângulo com três compartimentos: nome da classe, lista de atributos e lista de operações. Você também pode perceber a sintaxe de especificação dos atributos e operações.

Um atributo é especificado no seguinte formato:

[visibilidade] nome: tipo = valor_inicial

Visibilidade de um atributo diz respeito ao seu nível de acesso, ou seja, permite definir quais atributos de um objeto são acessíveis por outros objetos. Essa propriedade serve para implementar o encapsulamento da estrutura interna da classe. A visibilidade de um atributo pode ser:

Pública (+)

Somente as propriedades que são realmente necessárias ao exterior da classe devem ser definidas com visibilidade pública. Todo o resto é escondido dentro da classe por meio das visibilidades protegida e privativa.

Protegida (#)

A visibilidade protegida é visível para subclasses da classe em que o atributo foi definido.

Privativa (-)

A visibilidade privativa é invisível externamente à classe em que o atributo está definido.

De pacote (~)

Com visibilidade de pacote, o atributo é visível a qualquer classe que pertença ao mesmo pacote no qual está definida a classe.

Veremos a diante que um pacote é um agrupamento de diagramas UML. Veja, a seguir, a que correspondem os elementos nome e tipo do atributo:

Elemento nome

O elemento nome, única parte obrigatória na sintaxe do atributo, corresponde ao nome do atributo.

Elemento tipo

O elemento tipo, que especifica o tipo do atributo, é dependente da linguagem de programação na qual a classe será implementada.

Como exemplo de elemento tipo, podemos citar: int, float, boolean. No exemplo da Imagem 1, o atributo matrícula é do tipo int.

É possível declarar o valor inicial que o atributo assume quando um objeto daquela classe for instanciado.

> **Sempre que um objeto de uma classe é instanciado, o valor inicial declarado é automaticamente definido para o atributo.**

Em nosso exemplo, quando um novo aluno é inserido no sistema, assumimos que seu estiloAprendizagem é aula prática.

Uma operação é especificada no seguinte formato:

visibilidade nome(parâmetros): tipo-retorno {propriedades}

O elemento nome corresponde ao nome dado à operação, em que normalmente usamos verbos que denotem a ação realizada por essa operação (no exemplo, assistirAula). Uma operação é ativada (executada) quando um objeto envia uma mensagem para outro, sendo essa mensagem o nome da operação.

Comentário

Do mesmo modo que os atributos, a visibilidade pode ser pública (+), privada (-), protegida (#) ou de pacote (~).

Veja a seguir a descrição e algumas características dos parâmetros de uma operação:

Definição de parâmetro

Os parâmetros de uma operação são informações que ela recebe do objeto que envia a mensagem para requisitar a execução da operação.

Quantidade de parâmetros

Uma operação pode ter zero ou mais parâmetros, que devem ser separados por vírgulas.

Tipo de parâmetro

Parâmetros têm seu tipo definido: tipo retorno representa o tipo de dados do valor retornado por uma operação.

Atividade 1

Na abordagem da orientação a objetos, o elemento central é a classe. Sobre os atributos de classes, podemos afirmar que

A descrevem os dados que deverão ser armazenados pelos objetos originados a partir dessa classe.

B são as atividades do negócio representadas nas classes.

C cada atributo de uma classe se relaciona com outros atributos de outras classes.

D todos os atributos de uma classe são do mesmo tipo de dados.

E um atributo só pode assumir um único valor em todos os objetos instanciados a partir da sua classe.

Parabéns! A alternativa A está correta.

Atributos são os dados de uma classe que assumem valores específicos nos objetos instanciados a partir dessa classe, dentro do conjunto de valores possíveis denominado domínio.

Visão geral do diagrama de classes

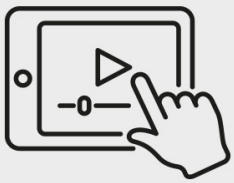
O diagrama de classes é uma ferramenta visual utilizada na modelagem de sistemas orientados a objetos, fornecendo representação abstrata das classes em um sistema e de seus relacionamentos e atributos. Além disso, ele oferece uma visão panorâmica da estrutura estática do sistema, destacando as entidades principais e suas interações.

Ao compreenderem a visão geral do diagrama de classes, desenvolvedores podem capturar de forma eficaz a arquitetura do sistema, identificar padrões de design e comunicar as relações entre os diversos componentes do software. Essa compreensão faz toda

diferença para o desenvolvimento de sistemas robustos e bem-organizados.

Você acompanhará neste vídeo, o diagrama de classes, uma ferramenta visual utilizada na modelagem de sistemas orientados a objetos, oferecendo uma visão panorâmica da estrutura estática do sistema.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



O diagrama de classes da UML é o modelo em que representamos as classes de um sistema, incluindo os detalhes sobre seus atributos e suas operações, e como essas classes se relacionam.

> **A elaboração do diagrama de classes é fundamental em todo o processo de desenvolvimento que segue a abordagem da orientação a objetos.**

Os diversos componentes de um diagrama de classes especificam as classes que serão realmente implementadas (ou seja, codificadas em uma linguagem de programação), sendo a base para a criação dos principais objetos e as interações entre eles.

Góes (2014) explica que o diagrama de classes é o modelo mais importante da UML, pois permite a representação dos elementos da base de dados do sistema. Portanto, facilita a comunicação entre a equipe de desenvolvimento e com os usuários finais. Ele também:

- Explicita as demandas de dados e informações dos atores, que são declaradas nos casos de uso.
- É usado para expressar graficamente os conceitos a serem manipulados por um sistema, bem como as ações esperadas.
- Fornece uma descrição independente de implementação dos tipos utilizados em um sistema, ou seja, os modelos de dados.

Costumamos dizer que esse diagrama destaca um aspecto estrutural estático do sistema, pois representa a estrutura interna do sistema, mas não como os objetos do sistema interagem no decorrer do tempo. As operações mostram quais são as possíveis interações entre os objetos, mas não explicitam em que momento serão invocadas.

O diagrama de classes é utilizado na construção do modelo de classes desde o nível de análise até o nível de especificação e evolui à medida que o processo de desenvolvimento do sistema avança.

O que queremos dizer quando afirmamos que o diagrama "evolui" à medida que o desenvolvimento do sistema avança?

Resposta

Isso significa que vamos acrescentando detalhes e refinando o diagrama, tanto em termos de atributos e operações quanto na forma como associamos as classes.

É comum chamarmos esse diagrama que evolui sucessivamente de modelo de classes de análise, modelo de classes de projeto e modelo de classes de implementação, fazendo referência às etapas do processo de desenvolvimento. Veja um pouco mais sobre esses modelos a seguir:

Modelo de classes de análise

É composto por elementos identificados na etapa de análise; é basicamente formado por classes relacionadas a conceitos do domínio da aplicação. Seu objetivo é descrever o problema representado pelo sistema a ser desenvolvido. É um modelo conceitual; com foco em o que o sistema deve tratar, não considera características técnicas da solução a ser utilizada. Em termos práticos, nesse modelo, incluímos as classes, a lista de

atributos e a lista básica de operações, sem os detalhes de tipos de dados, ou parâmetros das funções, por exemplo. O modelo de casos de uso e o modelo de classes de análise são os dois principais modelos criados na fase de análise.

Modelo de classes de projeto

É construído na atividade de projeto do desenvolvimento e considera alguns detalhes da solução de software a ser utilizada, mas ainda em um nível alto de abstração. Nessa etapa do processo de desenvolvimento, é comum percebermos a necessidade de criar novas classes, uma vez que o foco agora está em como o sistema deve funcionar. Ou seja, como o sistema vai entregar suas funcionalidades. Os detalhes incluídos são relacionados, por exemplo, com tipos e domínio dos atributos; parâmetros, entradas e saídas das operações etc. Novas classes criadas podem ser de interfaces do sistema, bancos de dados, entre outras.

Modelo de classes de implementação

É a codificação das classes definidas nas fases de análise e projeto em alguma linguagem de programação, normalmente uma linguagem orientada a objetos (por exemplo, C++, Java, C# etc.). O modelo de implementação é construído na atividade de codificação de um processo de desenvolvimento.

O exemplo da Imagem 2, extraído da obra de Bezerra (2015), ilustra a representação de uma classe em diferentes níveis de abstração. Observe que, em cada iteração do processo de desenvolvimento, são acrescentados mais detalhes sobre os atributos e operações:

Imagem 2: Exemplo de classe ContaBancária em diferentes níveis de abstração.

Na especificação mais detalhada da classe ContaBancária do exemplo da Imagem 2, você pode observar a presença dos símbolos – e + antes dos nomes dos atributos e das operações, que representam sua visibilidade.

Nessa mesma representação, temos as operações com seus parâmetros de entrada especificados (a Quantia em creditar e debitar).

Atividade 2

O diagrama de classes é uma representação visual das classes de um sistema e de seus relacionamentos, mostrando atributos, métodos e interações entre as classes, facilitando a compreensão da estrutura e do design do software durante o desenvolvimento.

Assinale a opção que apresenta uma afirmação verdadeira em relação às funções do diagrama de classes na UML.

A Serve apenas para representar as operações possíveis entre os objetos do sistema.

B Busca somente detalhar os atributos das classes do sistema.

C Atua apenas para facilitar a comunicação entre a equipe de desenvolvimento e os usuários finais.

D Expressa textualmente os conceitos a serem manipulados pelo sistema.

E Serve para especificar os elementos da base de dados do sistema, entre outras coisas.

Parabéns! A alternativa E está correta.

O diagrama de classes da UML é utilizado para representar as classes de um sistema, incluindo detalhes sobre atributos e operações, e como essas classes se relacionam. Além disso, ele permite a representação dos elementos da base de dados do

sistema. As outras opções restringem a abrangência das funções (A, B e C) ou apresentam uma função que não é verdadeira (D).

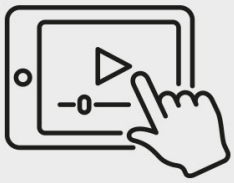
Relacionamentos no diagrama de classes

São fundamentais para representar as interações entre as classes de um sistema orientado a objetos. Além disso, os relacionamentos no diagrama de classes descrevem como as classes estão conectadas umas às outras e ajudam a definir a estrutura e o comportamento do sistema. No contexto da modelagem UML, os principais tipos de relacionamentos incluem associação, agregação, composição, herança e dependência.

Compreender esses relacionamentos ajuda a desenvolver um modelo preciso e coerente do sistema, permitindo que os desenvolvedores identifiquem e capturem de maneira eficaz as dependências e interações entre as diferentes partes do software.

Neste vídeo, você vai conferir as classes que se relacionam com outras classes, pois os objetos originados a partir delas trocam informações uns com os outros.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Você já viu que especificar uma classe diz respeito a definir seus atributos e operações. Mas classes não existem isoladas, elas só fazem sentido quando se relacionam a outras, pois os objetos originados a partir delas trocam informações uns com os outros; e nisso reside a implementação de um sistema orientado a objetos.

Dizemos que os objetos colaboram uns com os outros. O desenho do diagrama de classes mostra como os objetos instanciados a partir dessas classes podem se relacionar. Existem diferentes tipos de relacionamentos possíveis, bem como diversos detalhes que podem ser

acrescentados no diagrama para melhorar sua semântica. Trataremos deles a seguir.

> **A forma de relacionamento mais comum é chamada de associação e é representada no diagrama de classes por uma linha (normalmente um segmento de reta) ligando as classes às quais pertencem os objetos relacionados.**

A Imagem 3 adiante ilustra associações entre classes: Cliente faz Pedido; Professor ministra Disciplina.

Imagem 3: Associações simples entre classes.

Mas qual é o significado dessas associações? Quando ligamos as classes Cliente e Pedido significa que, durante a execução do sistema, haverá a possibilidade de troca de mensagens entre objetos dessas classes.

As associações possuem diversas características que podem ser representadas, por escolha do analista, para prover um melhor entendimento do modelo. Vamos conhecê-las a seguir:

Nome

Conforme a Imagem 3 mostra, os nomes das associações (ministra, faz) descrevem a relação que é estabelecida entre os objetos das classes associadas. Normalmente, são usados verbos ou expressões verbais para nomear associações.

Multiplicidades

As associações permitem representar a informação dos limites inferior e superior da quantidade de objetos aos quais outro objeto pode estar associado: são as chamadas multiplicidades.

Cada associação em um diagrama de classes possui duas multiplicidades, uma em cada extremo da linha que a representa. Os símbolos que representam uma multiplicidade são apresentados na tabela a seguir:

Apenas 1	1..1 (ou 1)
Zero ou Muitos	0..* (ou *)

Um ou Muitos	1..*
Zero ou Um	0..1
Intervalo Específico	li..ls

Tabela: Multiplicidades em um diagrama de classes.
Flavia Maria Santoro.

Na Imagem 4 a seguir, mostramos o exemplo com as multiplicidades correspondentes. Esses novos diagramas expressam mais do que o anterior: eles informam que um professor pode ministrar nenhuma (zero) e no máximo três disciplinas (0..3). Uma disciplina precisa ser ministrada por pelo menos um professor, mas pode ser ministrada por muitos (1..*).

Já um cliente pode fazer nenhum (zero) pedido, sem um limite máximo (0..*), e um pedido precisa ser feito por um cliente (1). Veja as multiplicidades:

Imagem 4: Associações simples entre classes com multiplicidades.

Tipo de participação

Indica a necessidade ou não da existência dessa associação entre objetos. A participação pode ser **obrigatória** ou **opcional**.

> **Se o valor mínimo da multiplicidade de uma associação é igual a 1 (um), significa que a participação é obrigatória. Caso contrário, a participação é opcional.**

A participação de Cliente e Professor nos exemplos da Imagem 4 é obrigatória, e a de Disciplina e Pedido é opcional. Isso significa que pode existir professores e clientes sem necessariamente estarem ministrando disciplinas ou fazendo pedidos.

Direção da leitura

A direção de leitura indica como a associação deve ser lida. Essa direção é representada por um pequeno triângulo posicionado próximo a um dos lados do nome da associação (como nos exemplos da Imagem 4).

Papéis

Quando um objeto participa de uma associação, ele tem um papel específico nela.

> **Uma característica complementar à utilização de nomes e de direções de leitura é a indicação de papéis (roles) para cada uma das classes participantes em uma associação.**

Atividade 3

Um diagrama de classes é uma representação gráfica que descreve a estrutura estática de um sistema, ajudando a visualizar e organizar a arquitetura do software durante o desenvolvimento. Nesse contexto, qual característica das associações no diagrama de classes permite representar a quantidade de objetos aos quais outro objeto pode estar associado?

A Nome.

B Multiplicidades.

C Tipo de participação.

D Direção da leitura.

E Papéis.

Parabéns! A alternativa B está correta.

A multiplicidade no diagrama de classes indica quantas instâncias de uma classe podem se relacionar com uma instância de outra

classe. Ela especifica os limites mínimo e máximo de relações, ajudando a definir a cardinalidade e as associações no sistema.

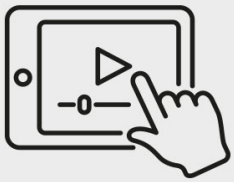
Abstrações aplicadas ao diagrama de classes

Ao lidar com classes associativas — isto é, as que estão ligadas a associações em vez de estarem ligadas diretamente a outras classes — ou classes que herdam características de outras, a utilização de abstrações se torna ainda mais importante. Por isso, as abstrações aplicadas ao diagrama de classes são importantes para simplificar a representação de sistemas complexos.

Assim, empregar abstrações no diagrama de classes para representar classes com características estruturais específicas permite que desenvolvedores foquem os aspectos essenciais da relação entre as classes, promovendo uma modelagem mais clara e eficaz do sistema.

Neste vídeo, você verá as abstrações que podem ser aplicadas ao diagrama de classes, incluindo as classes associativas, as autoassociações, a associação todo-parte e o mecanismo de herança.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Classes associativas

São classes que estão ligadas a associações, em vez de estarem ligadas a outras classes. São também chamadas de classes de associação.

Comentário

A classe associativa normalmente aparece quando duas ou mais classes estão associadas e há a necessidade de se manter informações sobre a associação que existe entre elas.

É muito comum a necessidade de classes associativas em relacionamentos muitos para muitos, ou seja 0..* 0..*.

Uma classe associativa é representada pela mesma notação utilizada para uma classe comum. A diferença é que aquela é ligada por uma linha tracejada a uma associação.

A Imagem 5 a seguir mostra uma classe associativa, na qual um professor ministra uma disciplina em determinada turma (que possui número, ano e semestre específicos).

Imagem 5: Exemplo de classe associativa.

Autoassociação

Tem como objetivo relacionar objetos da mesma classe, mas cada objeto tem um papel distinto nessa associação. Por exemplo, na Imagem 6 a seguir, objetos da classe Empregado estão associados entre si, um empregado como supervisor, que supervisiona outros empregados.

A imagem ilustra, também, o uso de papéis (roles) para distinguir o lado supervisor (multiplicidade 1) do lado supervisionado (lado muitos) da autoassociação.

Imagem 6: Exemplo de autoassociação.

Todo-parte

Existe uma categoria especial de relacionamento que possui uma semântica (significado) particular: todo-parte.

> **Uma relação todo-parte entre dois objetos indica que um dos objetos está contido (é uma parte do) no outro, ou seja, o objeto do lado todo contém o do lado parte.**

A UML define dois tipos de relacionamentos todo-parte: a **agregação** e a **composição**.

Atenção!

A agregação e a composição são casos especiais da associação. Logo, todas as características válidas para uma associação valem para agregações e composições. Agregações/composições são assimétricas no sentido que, se um objeto A é parte de um objeto B, o objeto B não pode ser parte do objeto A.

Agregações e composições propagam comportamento, pois um comportamento que se aplica a um todo, automaticamente se aplica às suas partes. Nas agregações/composições, as partes são normalmente criadas e destruídas pelo todo. Cada uma delas é representada da seguinte forma:

- Uma agregação é representada graficamente como uma linha que conecta as classes relacionadas, com um diamante (losango) branco perto da classe que representa o todo.
- Uma composição é representada por meio de um diamante negro.

A Imagem 7 apresenta um relacionamento de composição. Um pedido é composto de vários itens de pedidos.

Imagem 7: Exemplo de relacionamento de composição.

A diferença entre agregação e composição é sutil, pois não é muito bem definida na UML. Ambas são relacionamentos todo-parte, mas pode-se dizer que:

Agregação		Composição
Na agregação, a parte pode existir independentemente do todo.	×	Na composição, a existência da parte depende da existência do todo.

No exemplo da Imagem 7 não faria sentido a existência de ItemPedido se não existisse Pedido.

Exemplo

Como agregação, poderíamos citar a associação entre Equipe e Jogador, em que este último é parte da Equipe; entretanto, um Jogador existe independentemente da existência da Equipe. Isso significa que a destruição de uma Equipe (todo) não implicaria na destruição do Jogador (parte), como ocorre na composição.

Herança

Outro relacionamento com semântica específica é o de herança, ou relacionamento de generalização/especialização. Esse tipo de relacionamento, diferente dos que você estudou, significa um

relacionamento de generalidade ou especialidade entre as classes envolvidas.

Exemplo

O conceito animais é mais genérico que o conceito mamífero, que, por sua vez, é mais genérico que o conceito macaco.

Nesse tipo de relacionamento, as **classes mais específicas herdam características das classes mais genéricas**. Conheça a seguir alguns termos utilizados na herança:

Subclasse

Usamos o termo subclasse para denotar a classe que herda as propriedades de outra classe mediante uma generalização.

Superclasse

Já a classe que possui propriedades herdadas por outras classes é chamada de superclasse.

No diagrama de classes, a herança é representada por uma flecha partindo da subclasse em direção à superclasse.

> **Um relacionamento de herança representa um conjunto de objetos que compartilham um conjunto comum de propriedades (atributos, operações, associações), características inerentes à superclasse, além de possuírem características próprias.**

A Imagem 8 mostra um relacionamento de herança.

Imagem 8: Exemplo de relacionamento de herança.

Nesse exemplo, um professor pode ser do tipo Tempo Integral ou Horista. Ambos possuem nome e matrícula, porém, o Professor Tempo Integral possui salário, e o Professor Horista possui valor de hora paga.

Note que, além dos atributos da classe genérica (no exemplo, Professor), as classes especializadas herdam também as operações (não especificadas no exemplo), assim como os relacionamentos (no exemplo, a associação ministra) da classe genérica.

Atividade 4

Abstrações no diagrama de classes representam conceitos genéricos e essenciais do sistema, focando as características comuns das entidades, ajudando a simplificar a complexidade, organizando o sistema em níveis hierárquicos e facilitando a reutilização e a manutenção do código.

Qual das seguintes afirmações está correta em relação às abstrações classes associativas e à herança no diagrama de classes?

A — As classes associativas são representadas por uma flecha partindo da subclasse em direção à superclasse.

B — As classes associativas estão ligadas a relacionamentos entre outras classes por uma linha tracejada, representando uma associação.

C — A herança é representada por uma linha conectando as classes relacionadas, com um diamante branco perto da classe que representa o todo.

D — Na herança, as classes especializadas herdam apenas os atributos da classe genérica, e não as operações.

E — A herança é utilizada para relacionar objetos da mesma classe, mas com papéis distintos na associação.

Parabéns! A alternativa B está correta.

Classes associativas são aquelas que representam associações entre outras classes, sendo ligadas ao relacionamento dessas classes por uma linha tracejada, indicando sua relação.

Modelagem de classes na prática

Vamos ver como modelar o diagrama de classes a partir do diagrama de casos de uso, mais especificamente, das descrições de cada caso de uso. A descrição do caso de uso alugar veículo será utilizada para a identificação das classes de entidade do minimundo.

Classes de entidade são aquelas que representam os elementos do mundo real. São os elementos referente aos quais informações precisam ser tratadas e armazenadas pelo sistema. Porém, para que cada caso de uso seja executado, algumas classes de entidade são necessárias. A identificação de cada classe é feita da seguinte maneira:

- Os substantivos da descrição do caso de uso, que são as classes candidatas, devem ser sublinhados.
- Se cada substantivo sublinhado representar algo que possa ter mais de um representante no mundo real e se for necessário armazenar informações sobre cada representante, então esse substantivo passa de classe candidata a classe de entidade. Do contrário, ele é descartado.
- Outros substantivos sublinhados podem se referir às características dessas classes de entidade.
- Com a identificação de como as classes candidatas se relacionam, definindo nome do relacionamento e multiplicidade.

Vamos ver isso na prática!

Neste vídeo, você vai assistir à prática da modelagem de classes, que envolve a criação do diagrama que representa as classes de um sistema, seus atributos, métodos e relacionamentos a partir de um determinado minimundo, utilizando a UML.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Considerando a descrição do caso de uso alugar veículo apresentada, veja que ator funcionário executa caso de uso alugar veículo, de acordo com os seguintes passos:

1. Funcionário inicia o caso de uso.
2. Sistema solicita o código do funcionário.
3. Funcionário fornece seu código.
4. Sistema valida e solicita os dados do cliente.
5. Funcionário fornece a carteira nacional de habilitação (CNH) do cliente.
6. Sistema verifica a CNH.
    6.1. Se a CNH já for cadastrada, o sistema disponibiliza os demais dados (nome, telefone e endereço) e vai para o passo 7.
    6.2. Se não for, o funcionário fornece demais dados (Nome, telefone e endereço).
    6.3. Sistema armazena informações.
7. Sistema disponibiliza tipos de veículo de acordo com os automóveis à disposição para aluguel.
8. Funcionário seleciona tipo de veículo.
9. Sistema disponibiliza veículos daquele tipo.
10. Funcionário seleciona veículo, informando data de início e de fim de aluguel.
11. Sistema armazena dados do aluguel.
12. Caso de uso é encerrado.

Cenário de exceção 1

No passo 4, o código fornecido pelo funcionário no passo 3 não é validado pelo sistema:

- 4a. Sistema não valida código do funcionário.

- 4b. Se não for a terceira tentativa, o sistema retorna ao passo 3.
- 4c. Se for a terceira tentativa, o passo 12 é executado.

Cenário de exceção 2

No passo 7, não há veículos disponíveis para aluguel:

- 7a. Passo 12 é executado.

No caso de uso alugar veículo, temos o substantivo funcionário (que também é um ator) citado nos passos 1, 2, 3, 5, 6b, 8 e 10. Em relação ao funcionário, as duas perguntas do item 2 têm resposta positiva. Então, funcionário é uma classe de entidade. Repetindo os passos 2, 3 e 4 para todos os substantivos sublinhados, obtemos as classes de entidade funcionário, cliente, aluguel, tipo de veículo e veículo.

Por que essas classes? Porque elas representam elementos do mundo real referentes aos elementos dos quais o sistema deve armazenar informações. Como atributos (características) de cada classe, temos os demais substantivos: código do funcionário; CNH, nome, telefone e endereço do cliente e data de início e fim do aluguel.

Mas lembre-se: nem todos os atributos estarão explicitamente citados nas descrições. Como relacionamentos, veja as possibilidades que podemos inferir:

1. Um funcionário cadastra um ou vários aluguéis (o que só pode ser realizado por um funcionário).
2. Um cliente pode alugar vários veículos (um por vez) e que cada automóvel pode ser alugado por vários clientes (um por vez também). Portanto, o aluguel é uma classe associativa entre cliente e veículo.
3. Um veículo sempre é de um determinado tipo, que, por sua vez, tem vários veículos associados.

O processo realizado permite desenhar o diagrama de classes da figura. Ao fazer isso para cada caso de uso, conforme o processo descrito, as classes de entidade podem se repetir e novas classes e novos relacionamentos podem surgir. Ao final da análise de todos os casos de uso do diagrama, cria-se um diagrama final, unindo-se todos os diagramas criados, sem repetir classes e relacionamentos.

No caso de uso em questão, o diagrama criado é o da imagem a seguir. Para que ele represente também o diagrama de classes final, está

faltando apenas o relacionamento finalizar entre aluguel e funcionário, que representa que um funcionário pode finalizar vários objetos da classe aluguel, mas um aluguel só pode ser finalizado por um funcionário. Esse relacionamento é percebido no caso de uso encerrar aluguel. Veja!

Imagem 9: Diagrama de classes.

Atividade 5

Imagine que no caso de uso cadastrar novo veículo seja necessário que o funcionário forneça seu código. Assinale a alternativa que contenha as classes de entidade desse caso de uso.

A Funcionário, cliente e tipo veículo.

B Funcionário, veículo e tipo veículo.

C Veiculo, cliente e aluguel.

D Aluguel e veículo.

E Veiculo e cliente.

Parabéns! A alternativa A está correta.

As classes de entidade relacionadas a um caso de uso são as que referem às classes que o sistema deve armazenar e tratar informações. No caso específico desse caso de uso, as classes são: funcionário, cliente e tipo de veículo.



4 - Diagramas de objetos e de pacotes

Ao final deste módulo, você será capaz de empregar diagramas de objetos e de pacotes para apoiar a especificação de um sistema.

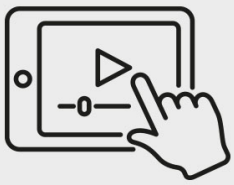
Visão geral do diagrama de objetos

Os diagramas de objetos, um dos elementos-chave da UML (Unified Modeling Language), oferecem uma perspectiva detalhada das instâncias das classes em um sistema. Similares aos diagramas de classes, eles retratam a estrutura estática do sistema, mas em um nível mais granular, mostrando instâncias específicas em determinado momento.

Esse tipo de diagrama facilita o entendimento das associações entre objetos, destacando os valores dos atributos e suas interações. Embora menos comuns que os diagramas de classes, os diagramas de objetos atuam na validação do sistema, oferecendo percepções valiosas durante o processo de desenvolvimento.

Você vai assistir, neste vídeo, ao emprego do diagrama de objetos na modelagem de objetos, que é o processo de representar entidades do mundo real como objetos no software, definindo suas propriedades e comportamentos e facilitando a implementação de sistemas orientados a objetos.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Além do diagrama de classes, a UML define um segundo tipo de diagrama na perspectiva estática e estrutural do sistema: o diagrama de objetos.

Comentário

Assim como objetos são instâncias de classes, os diagramas de objetos podem ser vistos como instâncias de diagramas de classes.

Um diagrama de objetos UML representa uma instância específica de um diagrama de classes em determinado momento. Portanto, do ponto de vista gráfico, ele é bastante parecido com o diagrama de classes. Bezerra (2015) afirma que um diagrama de objetos exibe uma “fotografia” do sistema em certo momento, mostrando as associações

estabelecidas entre objetos, de acordo com suas interações e com os valores que seus atributos assumem.

Um diagrama de objetos mostra:

- Os valores de atributos do conjunto de objetos que ele representa.
- Como esses objetos se relacionam entre si.

Um objeto, ou uma instância de uma classe, possui uma identificação única (não existem dois objetos iguais) e os seus atributos assumem determinados valores em um momento específico no tempo.

Os valores dos atributos podem sofrer alterações e, nesse caso, dizemos que o objeto sofreu uma **mudança de estado**.

Exemplo

O objeto Maria, criado a partir de uma classe Pessoa, possui como valor do atributo Endereço: Rua das Palmeiras. Quando Maria muda de endereço, o valor do atributo passa a ser o novo endereço: Avenida Vieira Souto. Assim, em um diagrama de estados, o objeto Maria seria representado com o valor do atributo Endereço quando o diagrama foi criado.

No diagrama de objetos, não faz sentido falar em operações, que são as mesmas para todos os objetos de uma classe. Portanto, nesse diagrama, o elemento principal, o Objeto, possui apenas o nome e os atributos com seus valores.

Cada objeto é representado por um retângulo com dois compartimentos. O compartimento superior mostra a identificação do objeto na forma de texto sublinhado:

:NomeClasse

nomeObjeto: NomeClasse

Lembre-se de que cada objeto é único e, portanto, esse nome identifica exclusivamente um objeto. A identificação do objeto pode ser omitida.

Exemplo

- Marta: Professor
- :Professor

No primeiro caso, o objeto da classe professor foi identificado com o nome Marta; no segundo, o analista optou por não caracterizar o objeto em si, significando que pode estar se referindo a qualquer objeto daquela classe.

Comentário

A representação com a identificação do objeto omitida é frequentemente usada na etapa de projeto do processo de desenvolvimento de sistemas, quando se representa a interação entre objetos (instâncias de classes) nos diagramas de sequência ou de comunicação.

No compartimento inferior (cuja utilização é opcional), aparecem os valores que os atributos definidos na classe assumem para este objeto. Dessa forma, deve mostrar uma lista de pares da forma: nome do atributo = valor do atributo.

A Imagem 1 adiante reproduz o exemplo de diagrama de objetos mostrado por Bezerra (2015). Nesse exemplo:

- Uma instância de pedido é associada a três instâncias de ItemPedido.
- Cada item, por sua vez, está ligado a um produto.

Os valores para cada atributo de cada objeto são apresentados nesse diagrama:

Imagem 1: Exemplo de dagrama de objetos.

Você pode observar também que não existem detalhes nos relacionamentos, tal como no diagrama de classes. Os relacionamentos no diagrama de objetos mostram uma situação específica em que aqueles objetos se encontram, não as possíveis relações entre eles.

No diagrama da Imagem 1, temos um Pedido que possui 3 Itens associados a ele, e 1 Produto associado respectivamente a cada Item.

A imagem 2, a seguir, mostra outro exemplo de diagrama de objetos, em que observamos que o Cliente Pablo está associado a 2 contratos de aluguel de carros diferentes. Cada contrato tem seu identificador e atributos próprios. Esse diagrama ajuda no entendimento da situação desse cliente:

Imagem 2: Exemplo de diagrama de objetos.

De acordo com Bezerra (2015), diagramas de objetos não são muito utilizados na prática. Mas eles podem ser úteis na atividade de validação no processo de desenvolvimento, pois permitem ilustrar e esclarecer certos aspectos de um diagrama de classes.

Dica

Mesmo antes de criar um diagrama de classes, você pode criar um diagrama de objetos para descobrir fatos sobre elementos específicos e suas associações ou para esclarecer dúvidas por meio de exemplos específicos. Além disso, ajuda na construção dos diagramas de interação (de sequência ou comunicação), pois estes utilizam a mesma notação.

Os diagramas de objetos podem ter as seguintes aplicações:

1

Ajudar na análise de uma interação específica.

2

Mostrar uma visão geral de alto nível do sistema.

3

Validar um diagrama de classes.

4

Verificar se o sistema foi desenvolvido conforme os requisitos.

5

Analisar se uma regra de negócio está sendo cumprida.

Para criar esses diagramas, procure exibir associações entre objetos somente com uma ligação (por exemplo, uma linha única juntando dois objetos, sem setas ou outros detalhes), pois um diagrama de objetos tem por objetivo mostrar somente a existência de associações e não os tipos de associações, como em um diagrama de classes.

Além disso, evite representar todos os objetos em um único diagrama de objetos, pois ele apresenta apenas um estado do objeto.

Atividade 1

Um diagrama de objetos é útil para entender como os objetos interagem durante a execução do software. Qual das opções a seguir destaca uma aplicação do diagrama de objetos da UML?

A Descrever os dados que deverão ser armazenados.

B Apoiar a análise de uma interação específica do sistema.

C Elaborar um novo diagrama de pacotes.

D Garantir que o sistema terá a performance desejada.

E Especificar os módulos do sistema.

Parabéns! A alternativa B está correta.

Diagramas de objetos podem, entre outras coisas, ajudar na análise de uma interação específica entre objetos do sistema.

Modelagem de objetos na prática

A geração de um diagrama de objetos dá a profissionais de desenvolvimento a possibilidade de validarem se o diagrama de classes criado realmente representa as situações encontradas no mundo real. Como exemplo para o exercício prático, será utilizado o mesmo caso de uso alugar veículo com o respectivo diagrama de classes.

A definição de um diagrama de objetos parte de um cenário que representa uma situação particular do minimundo. No exemplo, é apresentado o cenário no qual dois clientes alugam veículos de dois tipos diferentes com o mesmo funcionário. Vamos lá!

Você vai conferir, neste vídeo, a prática da modelagem de objetos, que permite representar entidades do mundo real como objetos no software, definindo suas propriedades e seus comportamentos a partir de um determinado minimundo, utilizando a UML.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Imagine a situação em que dois clientes alugam dois veículos de dois tipos diferentes, mas com o mesmo funcionário.

A primeira coisa a se fazer é identificar os objetos envolvidos. Nesse caso, temos os dois clientes (João e Maria), os dois veículos (V1 e V2) de um mesmo tipo (SUV média), o funcionário envolvido na operação (Paulo) e os dois aluguéis (A1 e A2). Tudo isso caracteriza oito objetos. Os passos para a criação do diagrama de objetos são os seguintes:

1. Criar a representação de um objeto para cada um dos objetos do mundo real.

2. Conectar as representações de objetos criadas seguindo o definido no diagrama de classes original.

3. Verificar a exatidão (se todas as conexões e objetos estão de acordo com diagrama de classes original) e a completude (se falta alguma conexão ou objeto, quer no diagrama de objetos ou no diagrama de classes), após fazer as conexões.

4. Averiguar as duas qualidades (completude e exatidão) permite que o diagrama de classes original seja ratificado e retificado, se for o caso.

O exemplo apresentado tem como consequência o diagrama de objetos da imagem. Verifique que todos os objetos foram representados e que as conexões (associações) apresentam as duas qualidades supracitadas.

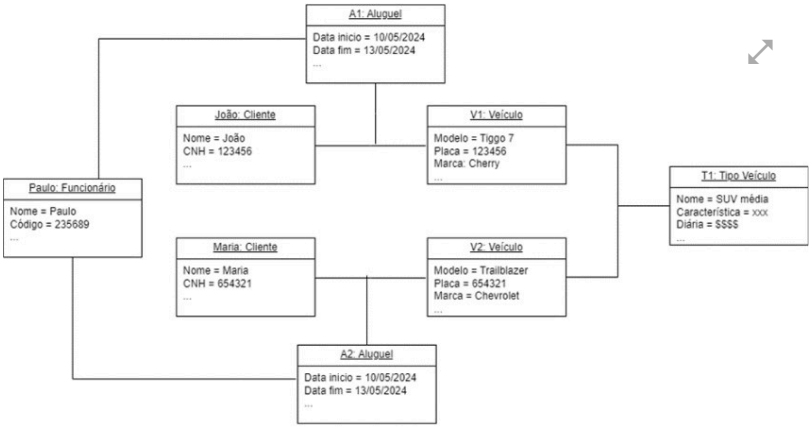


Imagem 3: Diagrama de objetos.

Atividade 2

O diagrama de objetos na modelagem de objetos representa instâncias de classes e suas interações em um sistema em tempo de execução. Nesse contexto, o que você, como profissional de engenharia de software, busca ao verificar a exatidão e completude das conexões em um diagrama de objetos?

A Validar se as mensagens estão representadas corretamente no diagrama de classes.

B Garantir que todos os objetos do mundo real estejam representados no diagrama de objetos.

C Assegurar que os objetos e suas conexões estejam de acordo com o diagrama de classes original.

D Verificar se as associações entre os objetos correspondem exatamente ao cenário do minimundo.

E Ratificar e retificar o diagrama de objetos para refletir com precisão as situações encontradas no mundo real.

Parabéns! A alternativa C está correta.

A verificação da exatidão e completude das conexões tem como objetivo garantir que os objetos e as conexões entre os objetos no diagrama de objetos estejam de acordo com o diagrama de classes original, ou seja, que correspondam às classes e associações definidas no diagrama de classes.

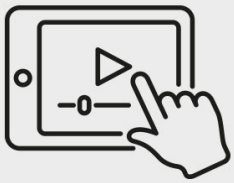
Diagrama de pacotes

Sistemas complexos exigem uma abordagem estruturada para sua representação, e os diagramas de pacotes da UML são uma solução eficaz para isso. Afinal, ao agruparem casos de uso, classes e outros pacotes, esses diagramas simplificam a visualização e organização do sistema.

Os agrupamentos feitos pelos diagramas de pacote da UML — também chamados de pacotes — permitem a definição de relacionamentos e visibilidades entre os elementos contidos, facilitando a compreensão da arquitetura do sistema. Além disso, os diagramas de pacotes são interessantes para ilustrar a interdependência entre os diferentes componentes do sistema e são amplamente utilizados na modelagem de casos de uso e na definição da arquitetura do software.

Você vai acompanhar, neste vídeo, o emprego do diagrama de pacotes, que permite organizar e agrupar elementos relacionados em unidades chamadas pacotes, facilitando a visualização da estrutura modular do sistema, mostrando dependências entre os pacotes e promovendo a organização e o gerenciamento eficientes do desenvolvimento de software.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Visão geral do diagrama de pacotes

Muitos sistemas possuem uma quantidade bastante grande de casos de uso e classes, portanto, representá-los em um único diagrama é inviável devido à sua complexidade para o entendimento. Para resolver esse problema, usamos o elemento da UML chamado de pacote (do inglês, *package*).

Os pacotes servem para agrupar diagramas, tais como:

- Classes
- Casos de uso
- Outros pacotes

Normalmente, os grupos de classes são associados com módulos de programação do sistema. Uma classe só pode aparecer uma vez no mesmo pacote, mas pode estar presente em vários pacotes diferentes.

A seguir, conheça algumas características do pacote:

Definição

Um pacote representa um grupo de elementos que pode se relacionar com outros pacotes por meio de uma relação de dependência.

Visibilidade

É possível definir a visibilidade, conforme estudamos anteriormente, de pacotes e dos elementos contidos nele.

Utilização

Pacotes são úteis para prover uma visão geral de um conjunto grande de objetos; para organizar um modelo grande e agrupar seus elementos.

A representação gráfica de um pacote é um retângulo com uma aba no canto superior esquerdo, com o nome do pacote na aba ou no centro do retângulo. A Imagem 4 adiante mostra um exemplo de pacotes que agrupam conjuntos de classes.

O pacote Módulo Docente agrupa as classes relacionadas com professores, o Módulo Discente agrupa as classes relacionadas com alunos, e o Módulo Organização Curricular agrupa classes pertinentes à matriz curricular, em um sistema acadêmico. Veja:

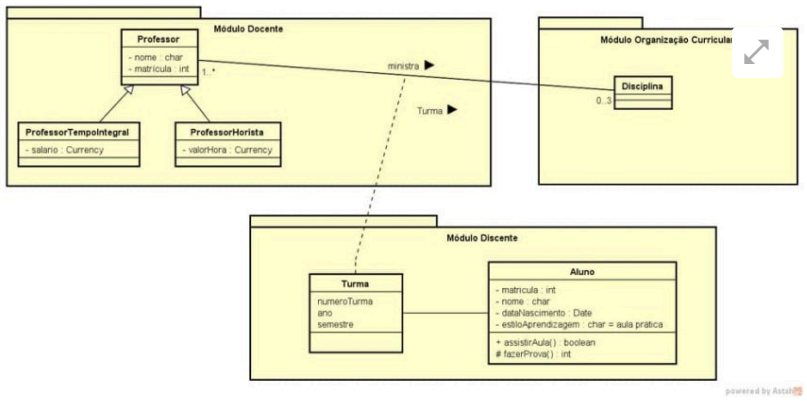


Imagem 4: Exemplo de representação de pacote.

Desdobramentos dos diagramas no pacote

As ligações entre pacotes são relacionamentos de dependência. Um pacote P1 depende de outro pacote P2, se algum elemento contido em P1 depende de algum elemento contido em P2.

A Imagem 5 adiante mostra o diagrama de pacotes do exemplo do sistema acadêmico sem os relacionamentos explícitos entre as classes internas; apenas com a representação de dependências existentes entre os próprios pacotes. Ou seja: os pacotes se relacionam com outros pacotes por meio de um vínculo de dependência. Veja:

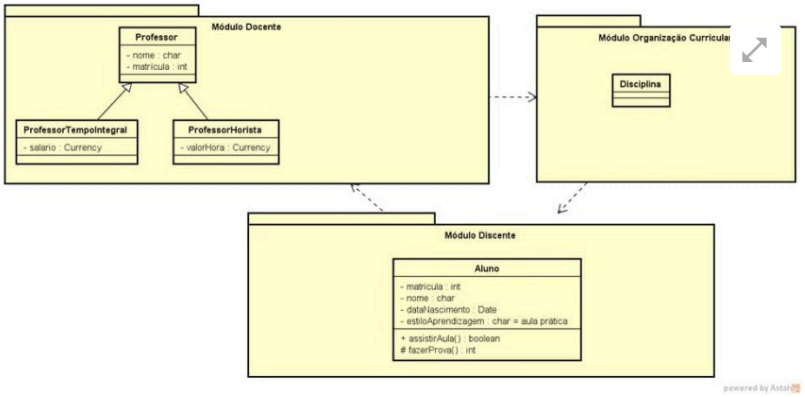


Imagem 5: Exemplo de representação de dependências entre pacotes.

As dependências entre pacotes mostram que um pacote depende ou influencia outro. Elas podem ser divididas em dois tipos:

Acesso

A dependência de acesso indica que um pacote requer assistência das funções de outro pacote.

Importação

A dependência de importação indica que uma funcionalidade foi importada de um pacote para outro.

Ambas são representadas por uma linha pontilhada com uma seta apontando para o pacote dependente.

Pacotes também podem formar grupos de pacotes e hierarquias. A Imagem 6 (Bezerra, 2015) mostra como pacotes podem ser agrupados dentro de outros pacotes, formando uma hierarquia de contenção:

Imagem 6: Agrupamento de pacotes.

Além disso, como já vimos, existem três tipos de visibilidade de elementos dentro dos pacotes:

+ (público)

Visível por todos que importam ou acessam o pacote.

(protegido)

Visível só pelos pacotes filhos (por relação de generalização).

– (privado)

Visível só por outros elementos do pacote.

Note que aqui não se aplica a visibilidade de pacotes (~), que é aplicável tão somente a atributo de classe que seja visível a qualquer classe pertencente ao mesmo pacote no qual está definida a classe.

No contexto da modelagem de casos de uso, os pacotes podem ser utilizados com diversos objetivos. Alguns desses objetivos são:

- Estruturar o modelo de casos de uso com base nos grupos de usuários do sistema.
- Definir a prioridade na qual os casos de uso serão desenvolvidos.
- Definir o grau de correlação entre os casos de uso.

O critério para criar os pacotes é subjetivo e depende da visão e das necessidades do projetista. Normalmente, o projetista define uma semântica e aloca elementos similares em um pacote, pois esses elementos tendem a ser alterados em conjunto no mesmo pacote.

Um pacote bem estruturado tem as seguintes características:

1

É coeso em relação aos elementos incluídos.

2

É fracamente acoplado em relação à exportação apenas dos

elementos que os outros pacotes precisam enxergar.

3

Não contém muitos alinhamentos, ou seja, possui um conjunto equilibrado de elementos

Ao especificar um pacote, é uma prática usar a forma simples de ícone retangular, obscurecendo o seu conteúdo. No caso da necessidade de apresentar o seu conteúdo interno (como nas imagens 4 e 5), deve-se mostrar apenas os elementos necessários para a compreensão do significado.

Esse diagrama também é muito utilizado para ilustrar a arquitetura de um sistema, mostrando o agrupamento de suas classes. A Imagem 7 adiante mostra um exemplo de diagrama de pacotes usado com o objetivo de organizar as classes de acordo com as camadas da arquitetura do sistema. Tipicamente, um sistema pode ser organizado conforme o padrão de projeto Model (Modelo), View (Visão) e Control (Controle).

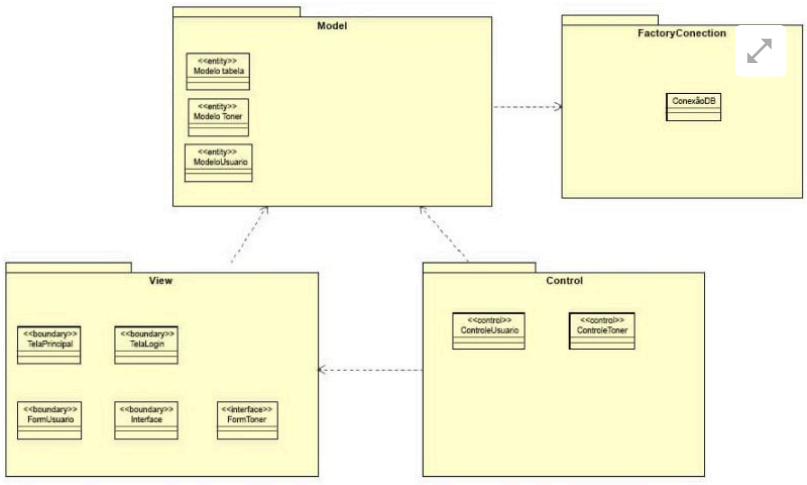


Imagem 7: Exemplo de diagrama de pacotes.

O pacote tem uma grande similaridade com a agregação. O fato de um pacote ser composto de modelos de elementos cria uma espécie de agregação de composição, pois, se esse pacote for destruído, todo o seu conteúdo também será.

Atividade 3

O diagrama de pacotes na modelagem de sistemas organiza elementos relacionados em unidades chamadas pacotes, facilitando a visualização da estrutura modular do sistema, mostrando dependências entre os pacotes e promovendo a organização e o gerenciamento eficientes do desenvolvimento de software.

Assinale a opção que apresenta um objetivo do pacote na UML.

A Agrupamento de diagramas.

B Representação da interação entre objetos.

C Especificação de classes.

D Permitir o polimorfismo.

E Agregar atributos de classes.

Parabéns! A alternativa A está correta.

Os pacotes servem para agrupar diagramas, tais como classes, casos de uso, ou mesmo outros pacotes.

Modelagem de pacotes na prática

A modelagem de pacotes, como parte integrante do processo de desenvolvimento de software, demanda uma compreensão profunda das estruturas e relações entre os componentes do sistema. Assim, por meio da prática, são desenvolvidas habilidades importantes, como a capacidade de dividir logicamente um sistema em módulos coesos, identificar dependências entre pacotes e organizar eficientemente a arquitetura do software. Além disso, a prática também facilita a colaboração entre equipes, permitindo uma divisão clara de responsabilidades.

Como exercício prático, veremos como o minimundo locadora de automóveis, usado como exemplo, poderia ser agrupado em pacotes. Vamos lá!

Você vai conferir, no vídeo, a prática da modelagem de pacotes por meio do diagrama de pacotes que representa a estrutura modular do software. Esse diagrama organiza elementos relacionados em pacotes, mostra dependências entre eles, facilita a gestão de complexidade e promove a modularidade e reutilização no desenvolvimento de sistemas a partir de um determinado minimundo, utilizando a UML.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

O minimundo utilizado, apesar de bem simples, pode ser usado para essa modelagem, pois os conceitos principais podem ser compreendidos independentemente do tamanho do exemplo. Para criar o diagrama de pacotes relacionado ao modelo de classes do exemplo, siga os passos a seguir:

1. **Identificação dos módulos principais**: analise o modelo de classes e identifique os principais grupos de classes que representam funcionalidades ou componentes relacionados. No exemplo, cliente, aluguel e veículo são os principais grupos lógicos.
2. **Agrupamento lógico**: reúna as classes em pacotes de acordo com a sua funcionalidade ou responsabilidade. Cada pacote deve conter classes que estejam relacionadas e que contribuam para uma parte específica do sistema. No exemplo, o pacote cliente contém a classe cliente; o pacote aluguel, as classes aluguel e funcionário; e o pacote veículo contém as classes veículo e tipo veículo.

3. **Definição de dependências**: identifique as dependências entre os pacotes, o que envolve determinar quais pacotes dependem de outros para funcionarem corretamente. Essas dependências podem ser representadas por setas que conectam os pacotes no diagrama. No exemplo, essas dependências são representadas pelas associações entre as classes pertencentes a cada pacote, como pode ser visto na Imagem 8 adiante.

4. **Organização hierárquica**: organize, se necessário, os pacotes em uma estrutura hierárquica, mostrando níveis de abstração ou modularidade no sistema. No caso em tela, temos que um aluguel só pode ser realizado se o veículo estiver previamente cadastrado. Se o sistema for expandido, podemos pensar um pacote funcionário para cadastro e em cadastro de clientes, havendo necessidade de criação de casos de uso respectivos.

5. **Refinamento e revisão**: revise o diagrama de pacotes para garantir que ele represente com precisão a estrutura e as relações entre os componentes do sistema, fazendo ajustes conforme necessário.

Por fim, observe a imagem 8, que representa o diagrama de pacotes para o roteiro de prática:

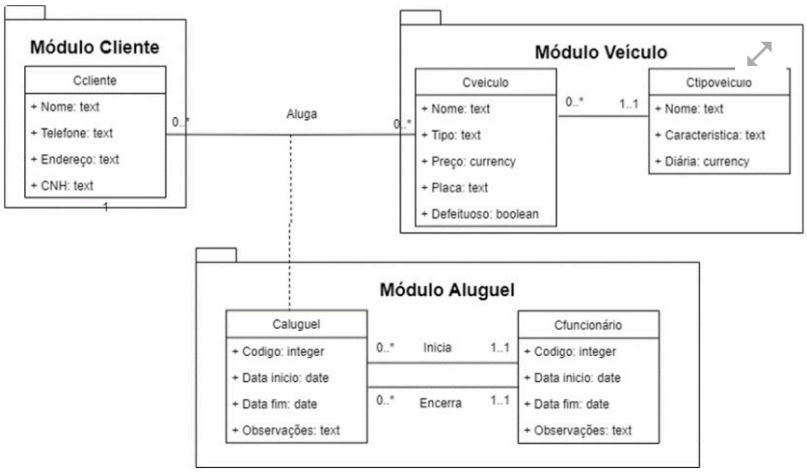


Imagem 8: Diagrama de pacotes.

Atividade 4

A modularidade no desenvolvimento de software proporciona benefícios significativos em termos de manutenção, reutilização, testes, colaboração, escalabilidade, flexibilidade e qualidade, contribuindo para

a criação de sistemas mais eficientes. Você, um analista de sistema finalizando o desenvolvimento do modelo de classes, vai para a fase seguinte de identificação dos principais módulos para criar um diagrama de pacotes vinculado ao modelo de classes de um sistema com qual objetivo?

A Definir a sequência de implementação dos pacotes do sistema.

B Agrupar as classes em pacotes com base em sua funcionalidade ou responsabilidade.

C Reconhecer as conexões entre os pacotes e classes do sistema.

D Especificar as relações entre as classes e pacotes do sistema para assegurar sua operação adequada.

E Examinar o modelo de classes e determinar os grupos de classes predominantes que representam funcionalidades ou componentes relacionados.

Parabéns! A alternativa E está correta.

A etapa de identificação dos principais módulos visa examinar o modelo de classes do sistema e determinar os grupos de classes predominantes que representam funcionalidades ou componentes relacionados. Essa análise inicial ajuda a estabelecer os pacotes iniciais que serão criados no diagrama de pacotes, proporcionando uma estrutura inicial para a organização das classes. As outras opções descrevem aspectos diferentes do processo de criação de um diagrama de pacotes, mas não abordam diretamente a identificação dos principais módulos com base no modelo de classes do sistema.

O que você aprendeu neste conteúdo?

- Requisitos funcionais e não funcionais de um sistema.
- Aplicação do diagrama de casos de uso na especificação de sistemas.
- Especificação textual de um caso de uso.
- Aplicação do diagrama de classes na especificação de sistemas.
- Uso dos diagramas de objetos e de pacotes na especificação de sistemas.

Podcast

Agora, escute o encerramento do tema, que destaca os pontos mais relevantes da modelagem do domínio, as boas práticas na especificação de requisitos, diagrama de casos de uso e diagrama de classes.

Para ouvir o *áudio*, acesse a versão online deste conteúdo.



Explore +

Busque o artigo **Regras do negócio, um fator chave de sucesso no processo de desenvolvimento de Sistemas de Informação**, dos professores Silvia Inês Dallavalle e Edson Walmir Cazarini, e aprenda

sobre a importância das regras de negócio no desenvolvimento de sistemas.

Referências

ASTAH. **Astah UML Modeling Tool**. Consultado na internet em: 31 jan. 2021.

BEZERRA, E. **Princípios de análise e projeto de sistemas com UML**. Rio de Janeiro: Elsevier, 2015.

GÓES, W.M. **Aprenda UML por meio de estudos de caso**. São Paulo: Novatec, 2014.

IEEE. **IEEE STANDARD GLOSSARY of Software Engineering Terminology**. *In:* IEEE Std. 610.12-1990, 31 dez. 1990. Consultado na internet em: 28 ago. 2024.

LIMA, A. S. **UML 2.0**: Do requisito à solução. São Paulo: Érica, 2005.

MEDEIROS, E.S. **Desenvolvendo software com UML 2.0**: definitivo. São Paulo: Pearson Markron Books, 2004.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

Material para download

Clique no botão abaixo para fazer o download do conteúdo completo em formato PDF.

Download material

O que você achou do conteúdo?

Relatar problema